

コンピュータ アーキテクチャ I 第6回

塩谷 亮太 (shioya@ci.i.u-tokyo.ac.jp)

東京大学大学院情報理工学系研究科 創造情報学専攻

課題の解説

課題 5

- 第4回の講義資料を参考に、P型/N型リレーを使って以下を構成せよ

- 2入力 OR ゲート
- 3入力 NOR ゲート

2入力 OR の真理値表

a	b	o
0	0	0
0	1	1
1	0	1
1	1	1

3入力 NOR の真理値表

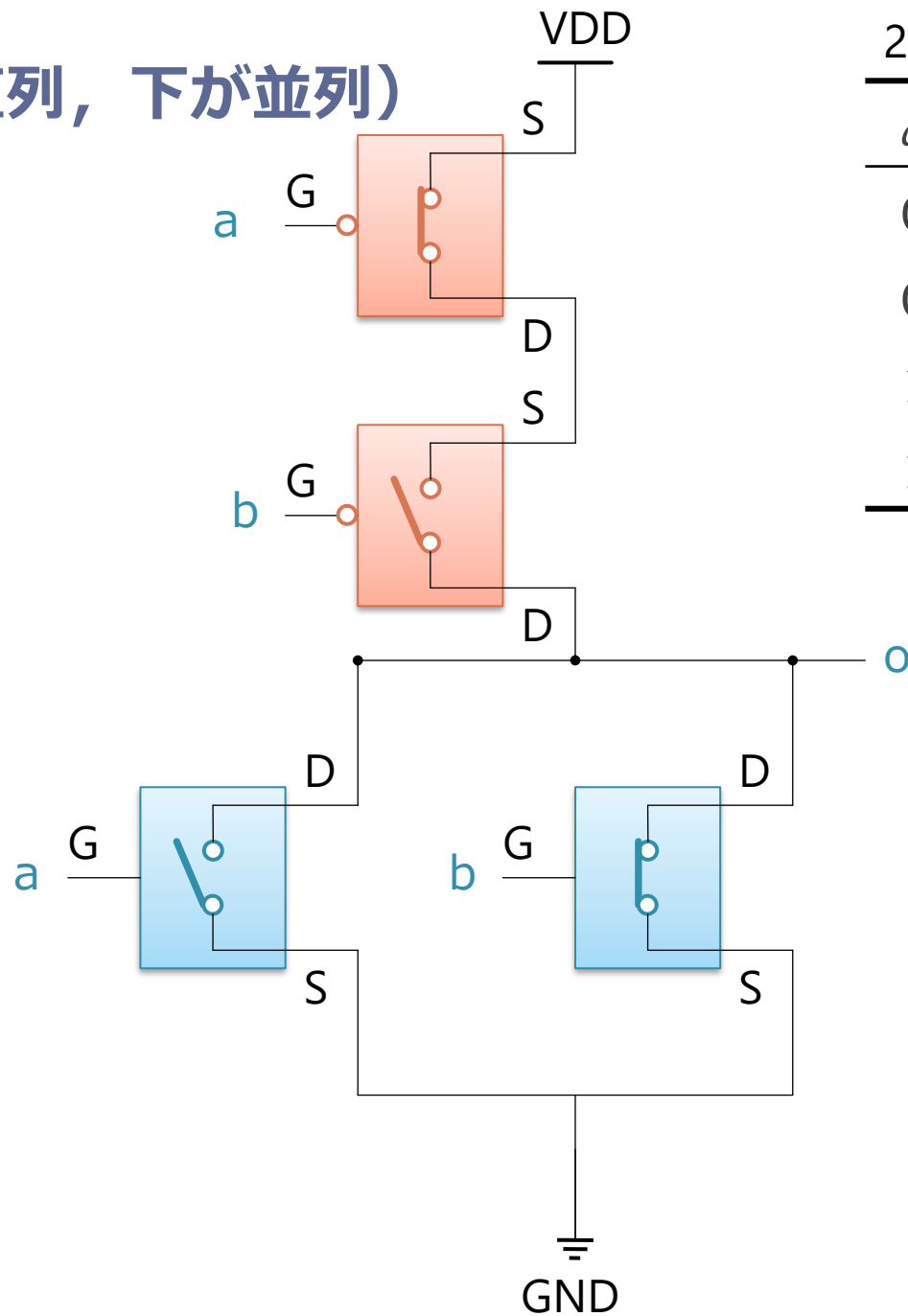
a	b	c	o
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

OR の作り方

- NOR を作って NOT で反転する
 - NAND を作って NOT で反転して AND を作ってたのと同じ

NOR

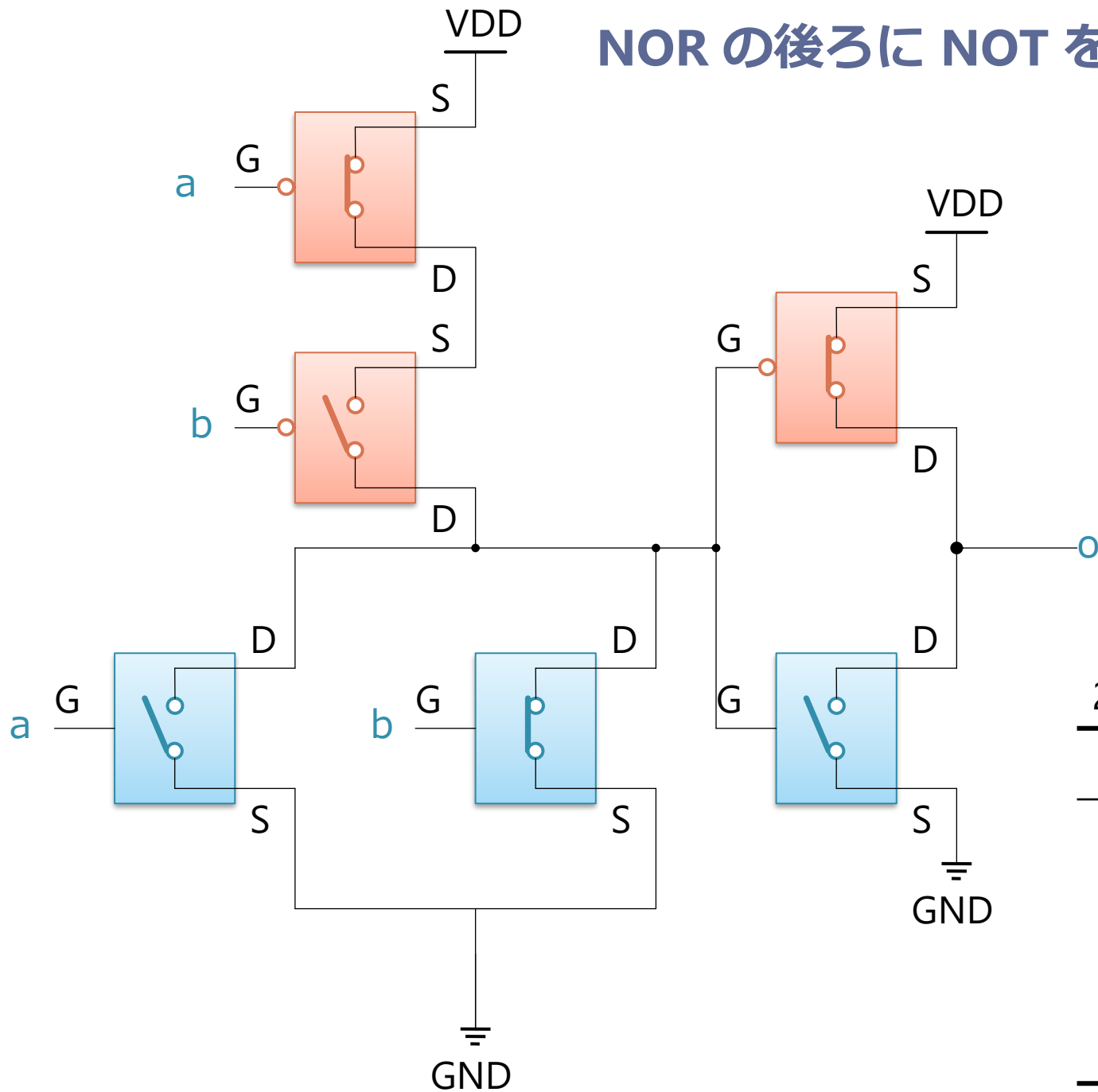
(上が直列, 下が並列)



2入力 NOR の真理値表

<i>a</i>	<i>b</i>	<i>o</i>
0	0	1
0	1	0
1	0	0
1	1	0

NOR の後ろに NOT を接続



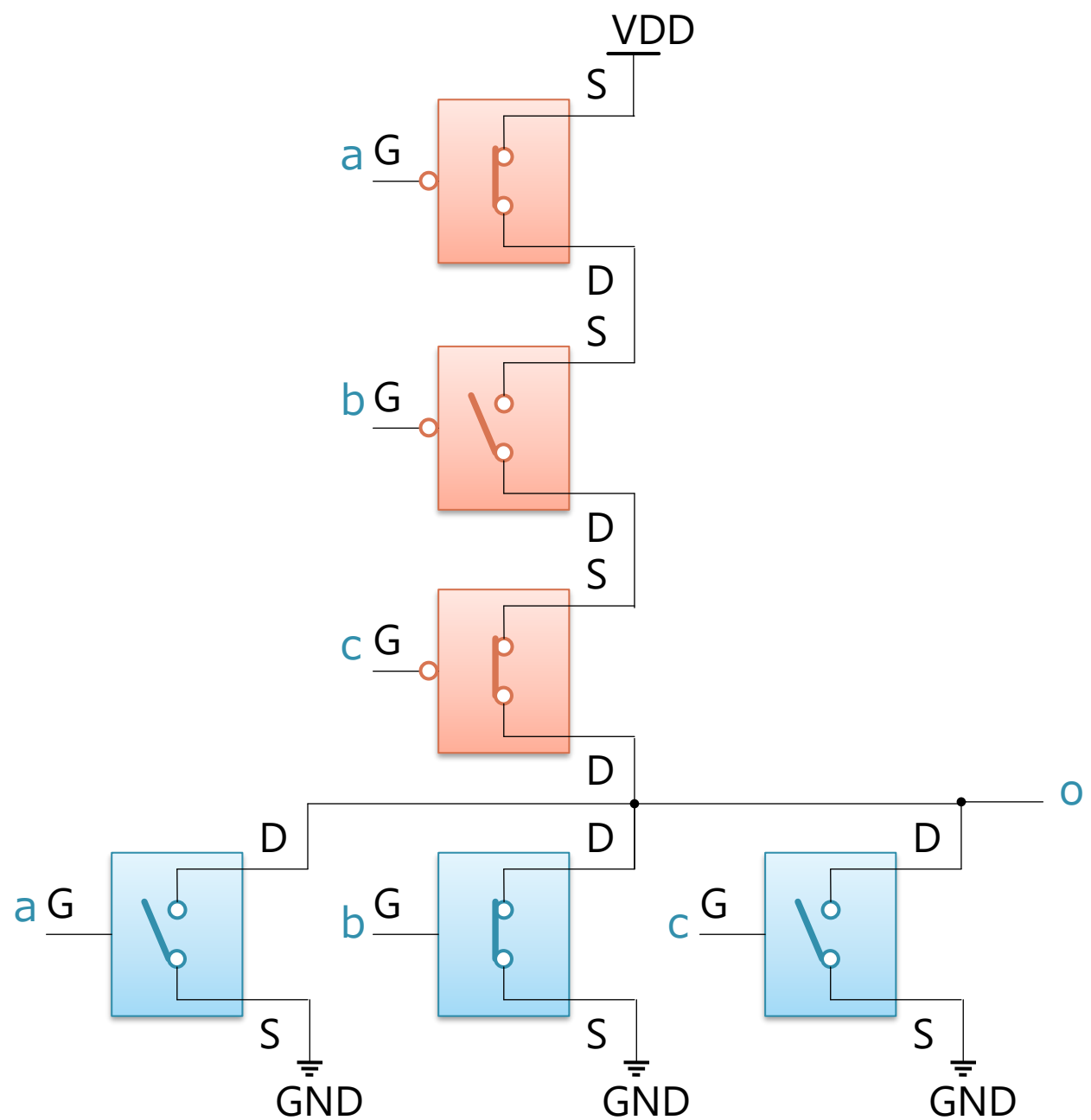
2入力 OR の真理値表

<i>a</i>	<i>b</i>	<i>o</i>
0	0	0
0	1	1
1	0	1
1	1	1

3入力NOR

3入力 NOR の真理値表

<i>a</i>	<i>b</i>	<i>c</i>	<i>o</i>
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

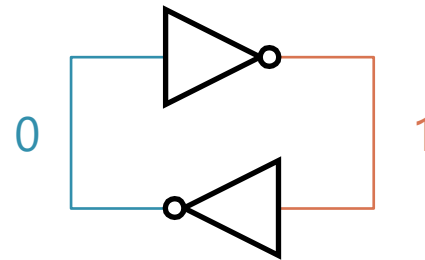
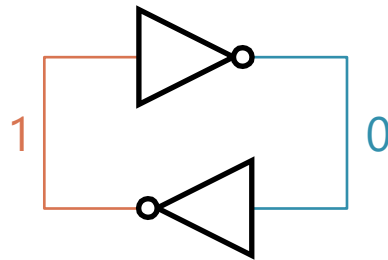


前回の振り返り

記憶素子の原理

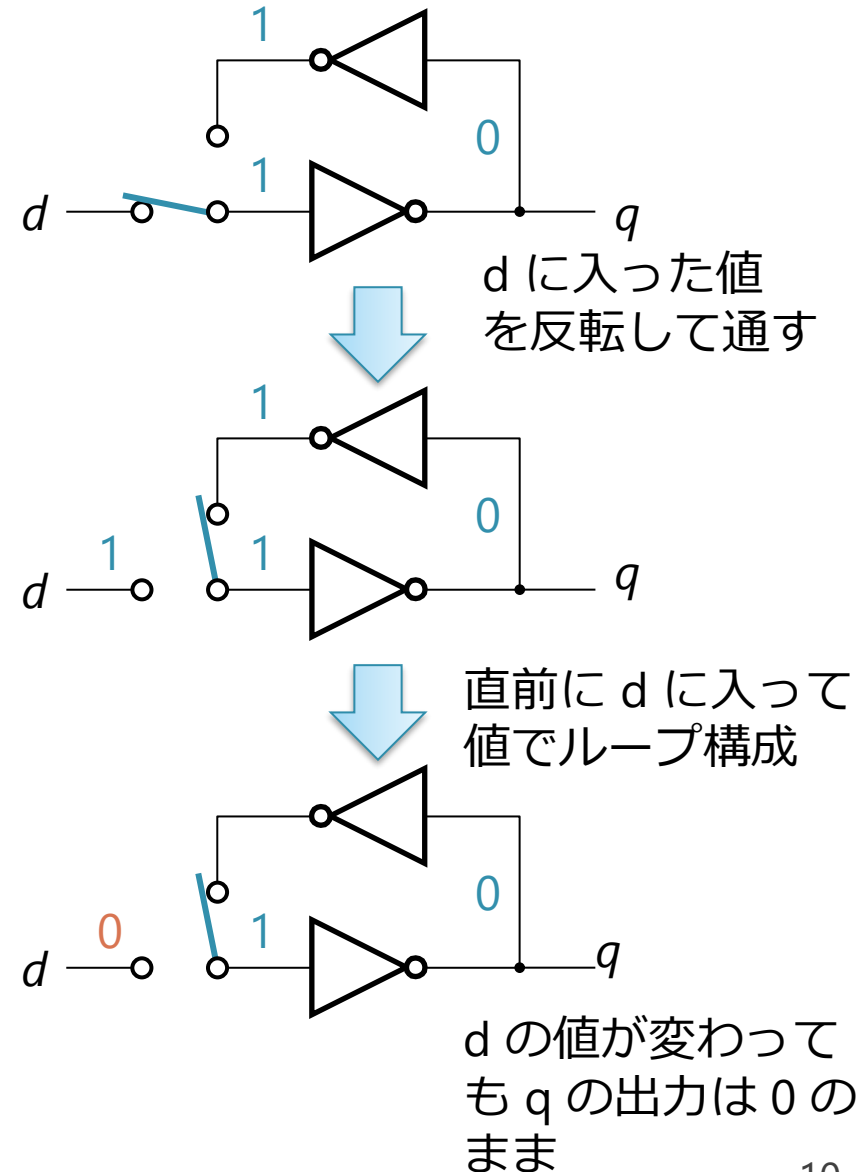
■ 記憶の実現方法：

- 2つの NOT ゲート（インバータ）をループさせた回路により実現
- 以下の2通りの安定状態がある
 - これのどっちになっているによって、1 bit の情報を記憶



D ラッチの回路

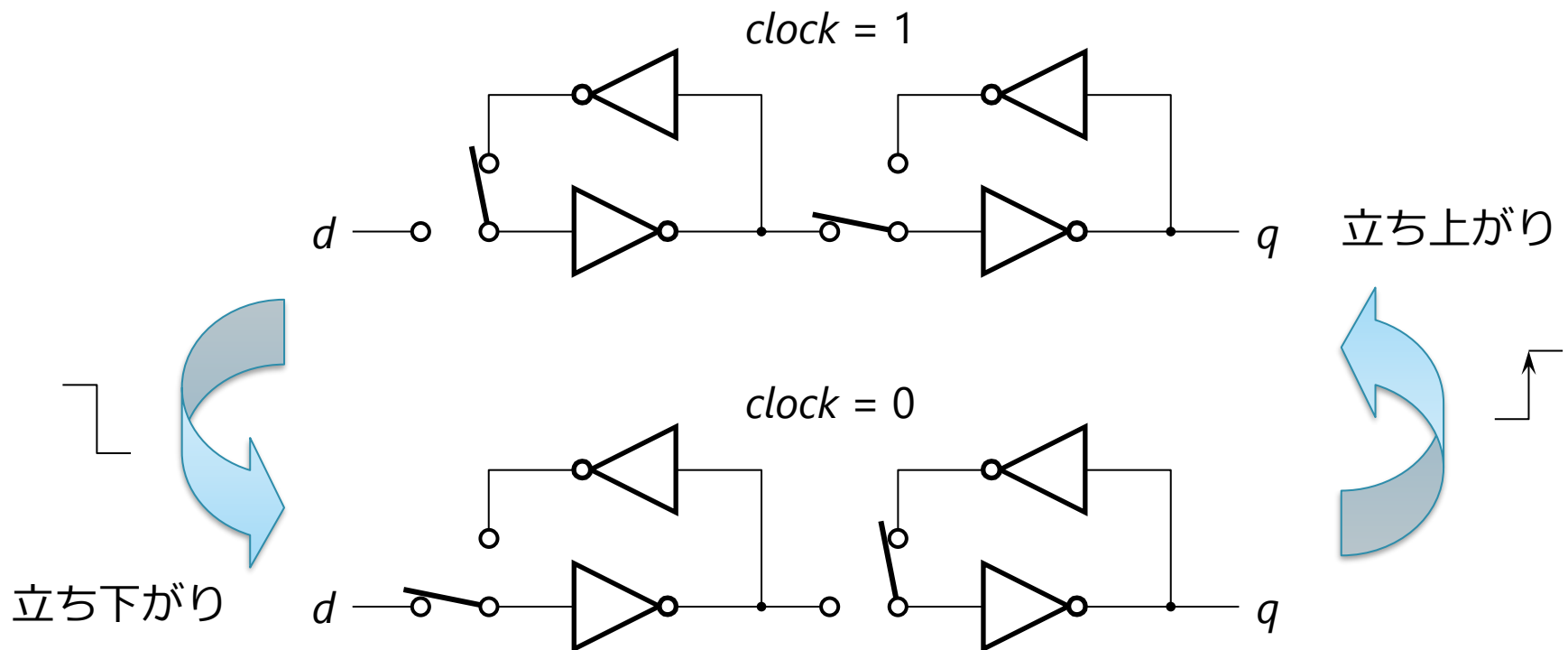
- 構造：マルチプレクサが入ったインバータのループ
 - ◇ ここではマルチプレクサを切り替えスイッチとして説明
 - ◇ クロックの立ち上がりのたびに、スイッチが切り替わる
 - ◇ d の値をループに取り入れ、取り入れた値が q から出力される



D-FF の回路

■ 構造：D ラッチを2つ繋げたもの

- ◇ D ラッチ 1 つだと，半周期は d に入った値が反転して素通しするので使いにくい
- ◇ 2 つ直列に繋げて素通しの期間をなくす

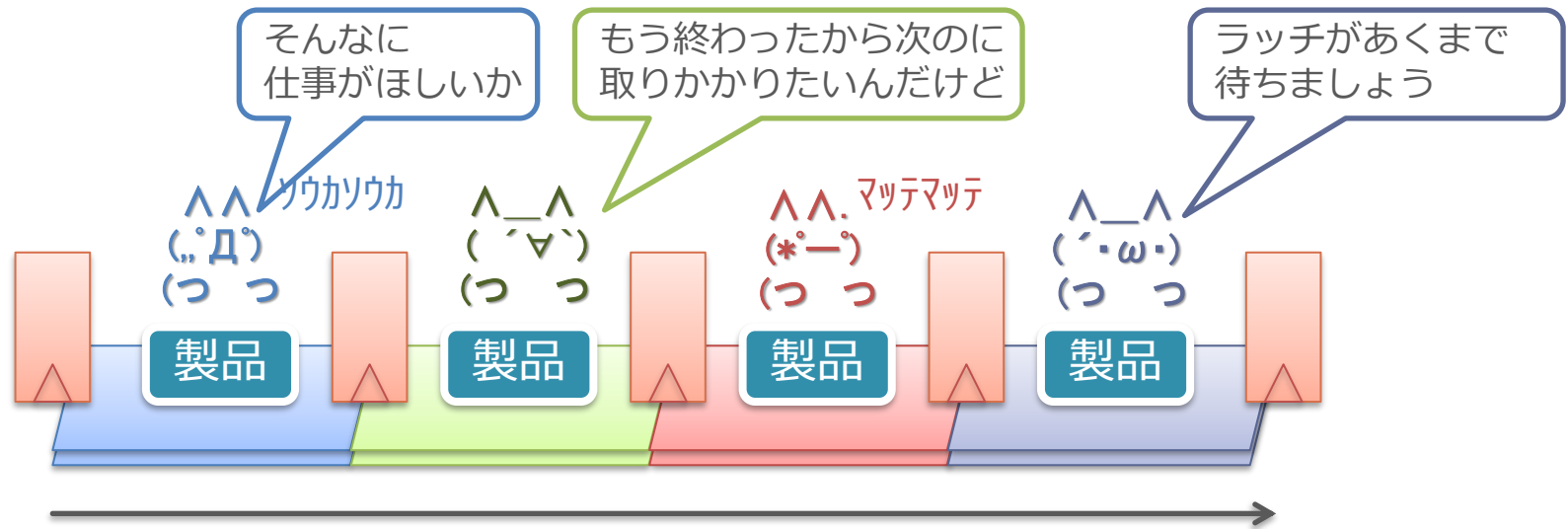


工場のラインを考える（再）



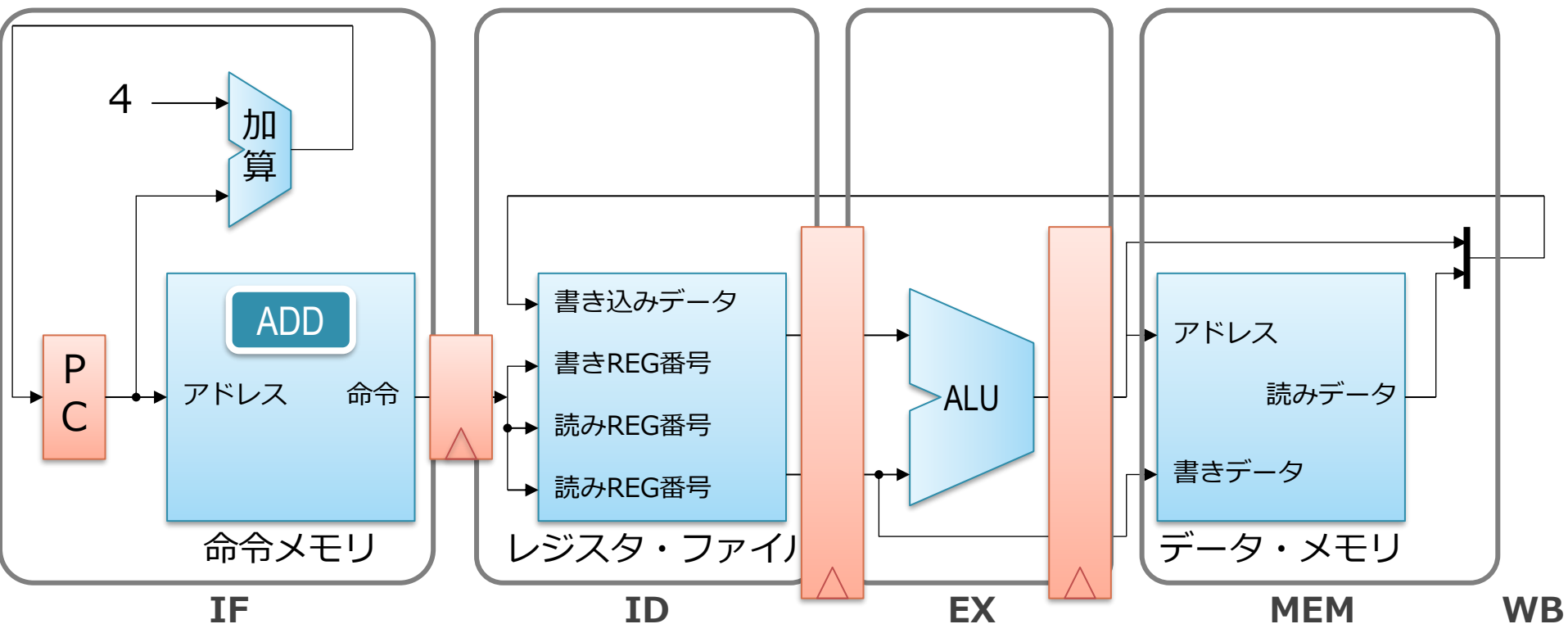
- 実際の工場：複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
- これが 命令パイプライン

パイプライン・ラッチのイメージ



- 各人の作業が終わっても，ラッチが開くまでは次の人に製品を送れない
 - 複数ステージ間で信号が混じるのを防ぐ
 - 指定された時間までラッチでドアを開かなくするイメージ？

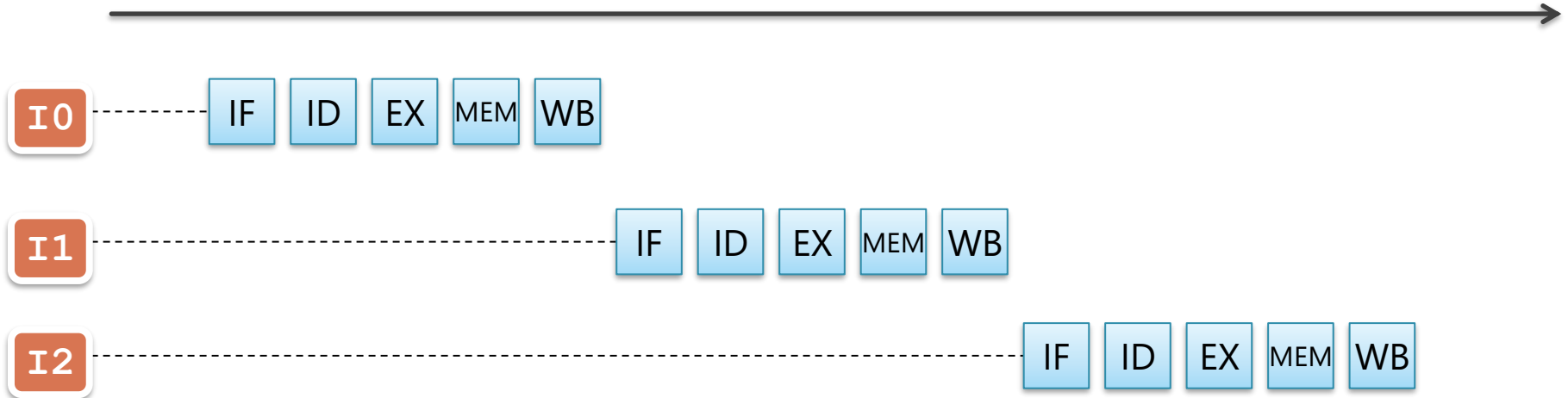
パイプライン化（オーバーラップ）の実現方法



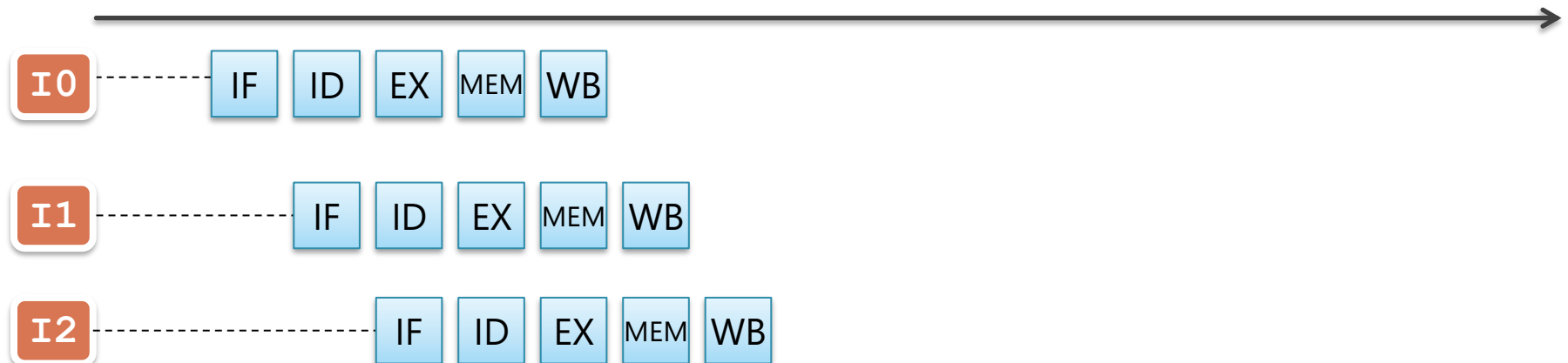
- 各ステージの間に, D-FF (オレンジの四角) を入れる
 - WB の書き込みについては, レジスタ・ファイル自体がクロックに同期して書き込みが行われるので D-FF は不要
- 各ステージの処理が早く終わっても, 次のクロックまでは D-FF で信号の伝搬を止める

パイプライン化による性能（スループット）向上

パイプライン化しない場合



パイプライン化した場合



パイプライン化の意味

- パイプライン化の効果：
 - スループットの向上
 - = 単位時間あたりに処理できる命令の数の増加
 - = 動作クロック周波数の向上
- これらは同じ事を言い換えてるだけ

パイプライン化の限界



- パイプライン段数を増やしていけば、どこまでも速くなるのか？
 - ならない
- 理由：
 1. 回路的な理由による周波数向上の限界（前回の講義で説明）
 2. **アーキテクチャ的な理由（ハザード）による実効性能の限界**

ハザード

ハザード (hazard)

- パイプラインがうまく動作しないこと
 - パタヘネ（教科書）の定義：
「パイプラインにおいて
次のサイクルに次の命令を実行できないこと」
- ハザードの種類：
 1. 構造ハザード
 2. 非構造ハザード
 1. データ・ハザード
 2. 制御ハザード

もくじ

1. 構造ハザード：ハード資源の不足に起因

1. 構造ハザードとはなにか？
2. その解決方法

2. 非構造ハザード：バックエッジに由来

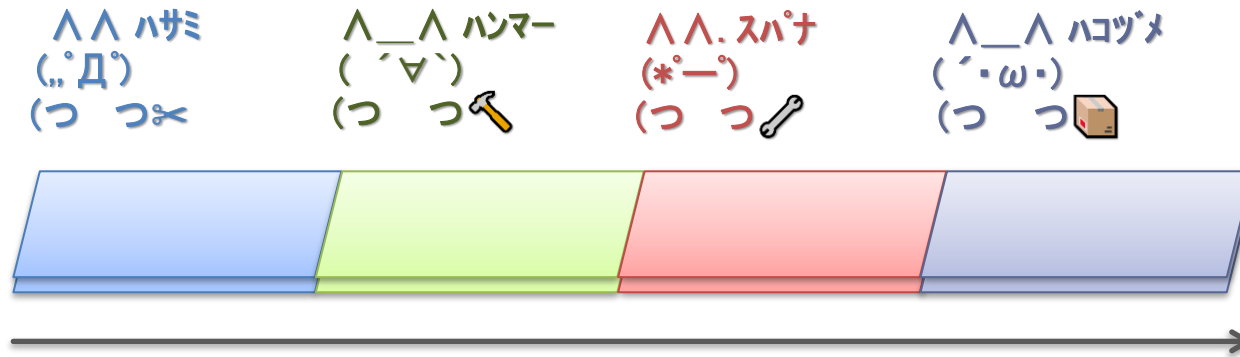
- a. データ・ハザード
- b. 制御ハザード

構造ハザード

構造ハザード

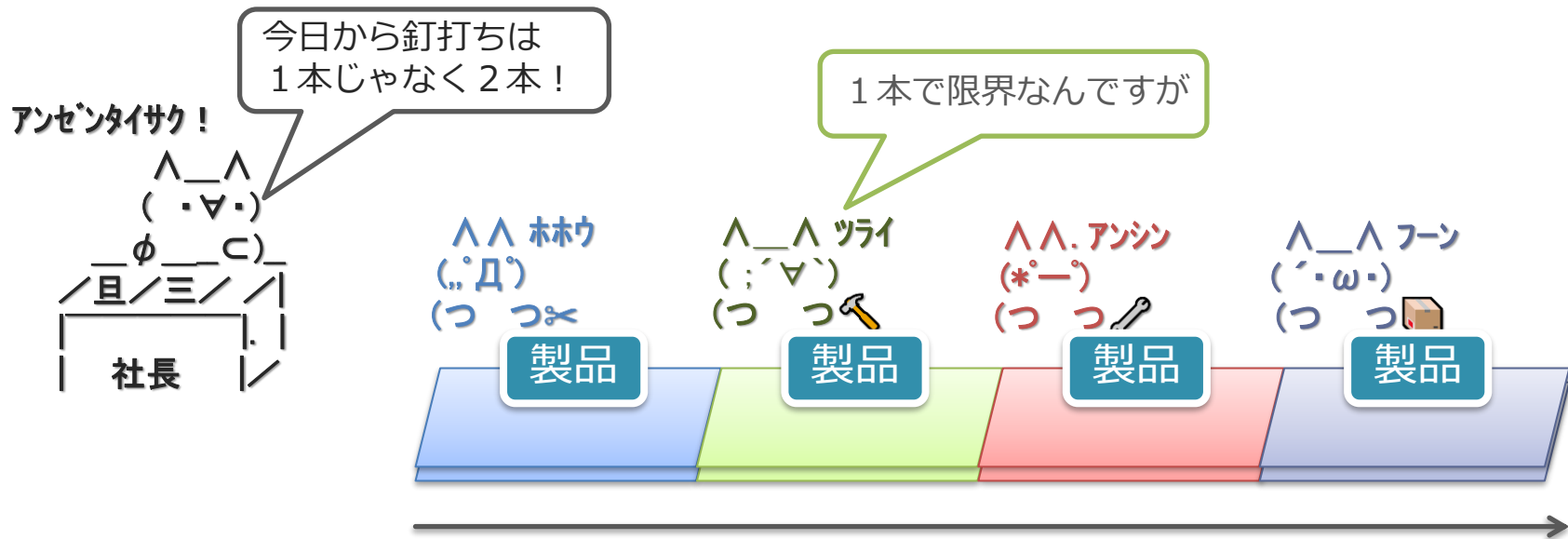
- ハード資源の不足により, パイプラインがうまく動作しないこと
- 工場のラインの例を使って説明

工場のライン



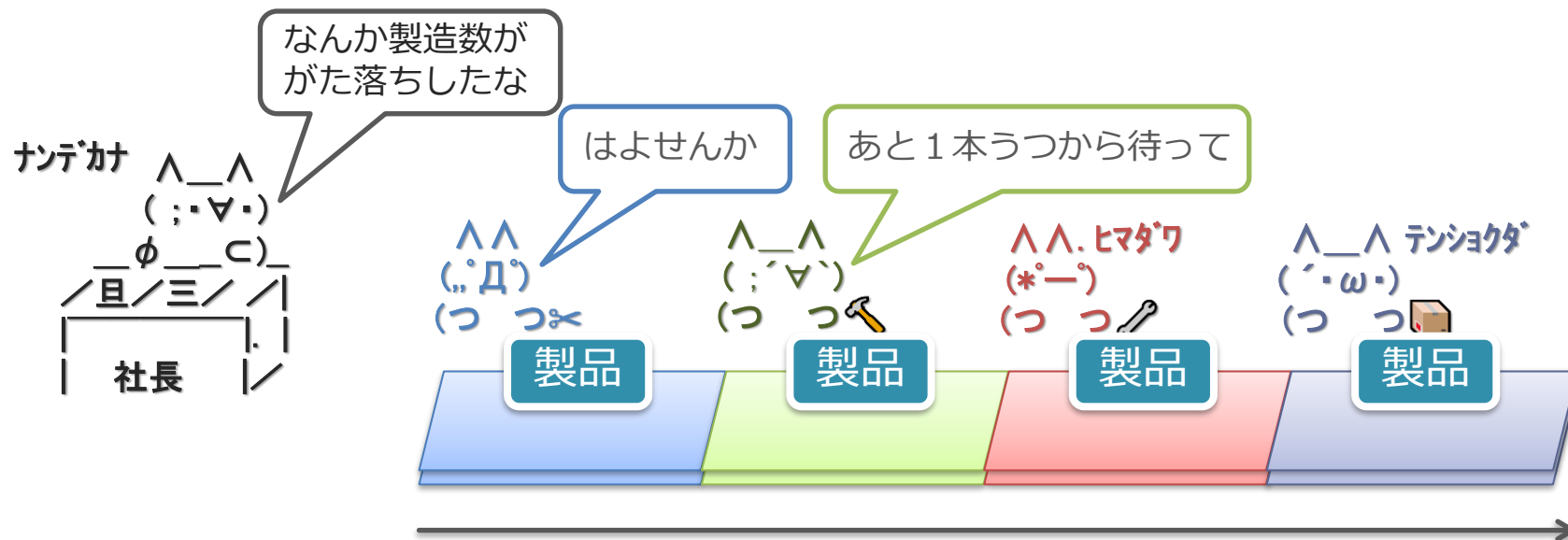
- 各ステージの担当者は製品が流れるまでの間に一定の仕事ができる
 - はさみで紙を1枚切る
 - ハンマーで釘を1本打つ
 - スパナでねじを1個しめる
 - 箱に製品を1つ入れる

釘の本数の変更



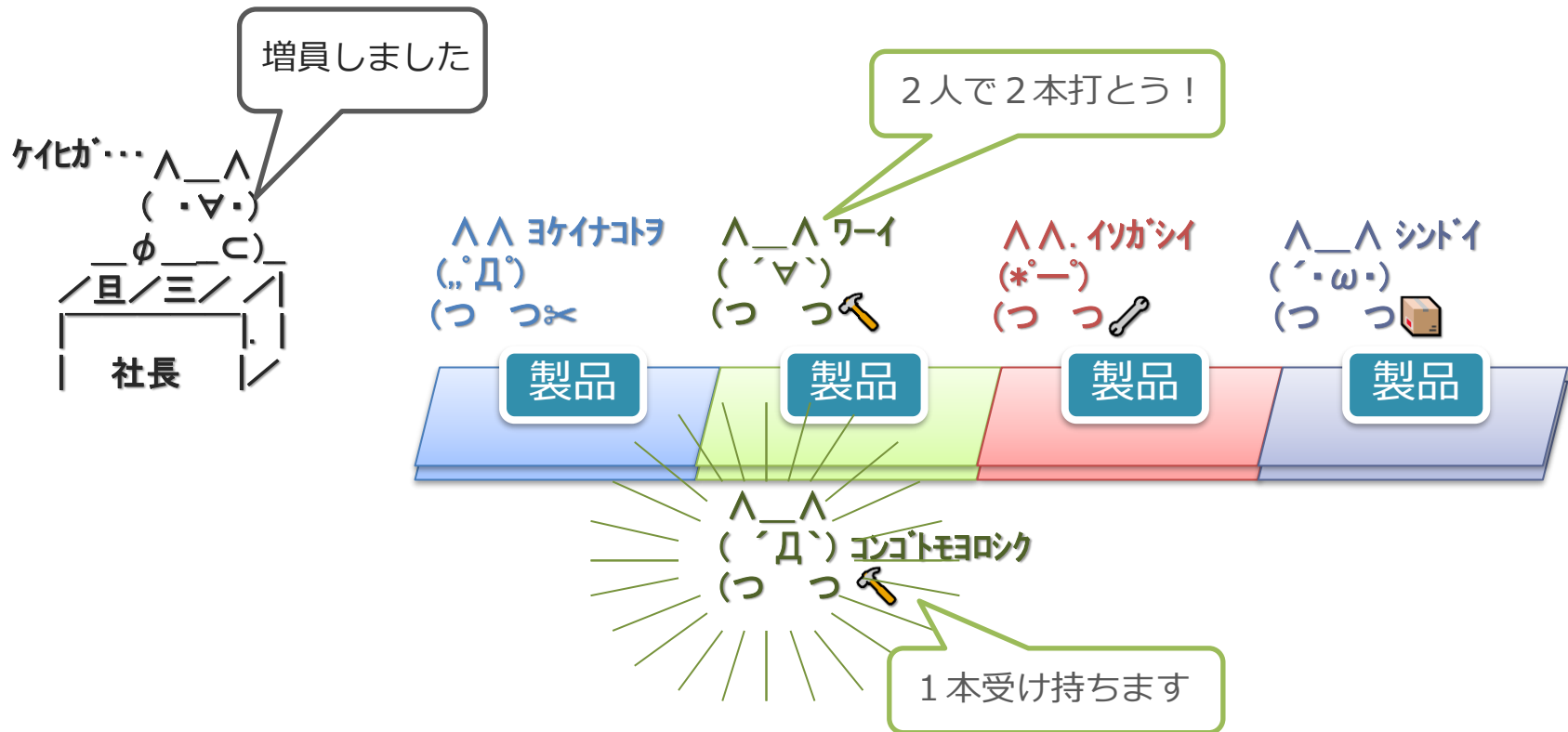
- 製品の安全対策のために釘の本数が1本から2本に
 - しかし, ハンマーを持っている釘打ち担当は単位時間に1本しか打てない

釘打ちの人のところで構造ハザード



- 2人目(; ' ∇ `) の人は単位時間に1本しか打てない
 - ラインが一番遅い人に合わせて動く（前回の講義より）
 - そこで2本打つまでライン全体が止まる
 - 全体の速度がそこで決まってしまう

増員による構造ハザードの解消



- 1人増やして緑のステージで単位時間に2本の釘が打てるように
 - これがハード資源の追加による構造ハザードの解消
 - ただし、追加しただけ経費がかかる
 - (ハードだとその分複雑になって電力を食う)

コンピュータにおける構造ハザード

■ ハード資源：

- 演算器（FU）, レジスタ・ファイル, メモリ など

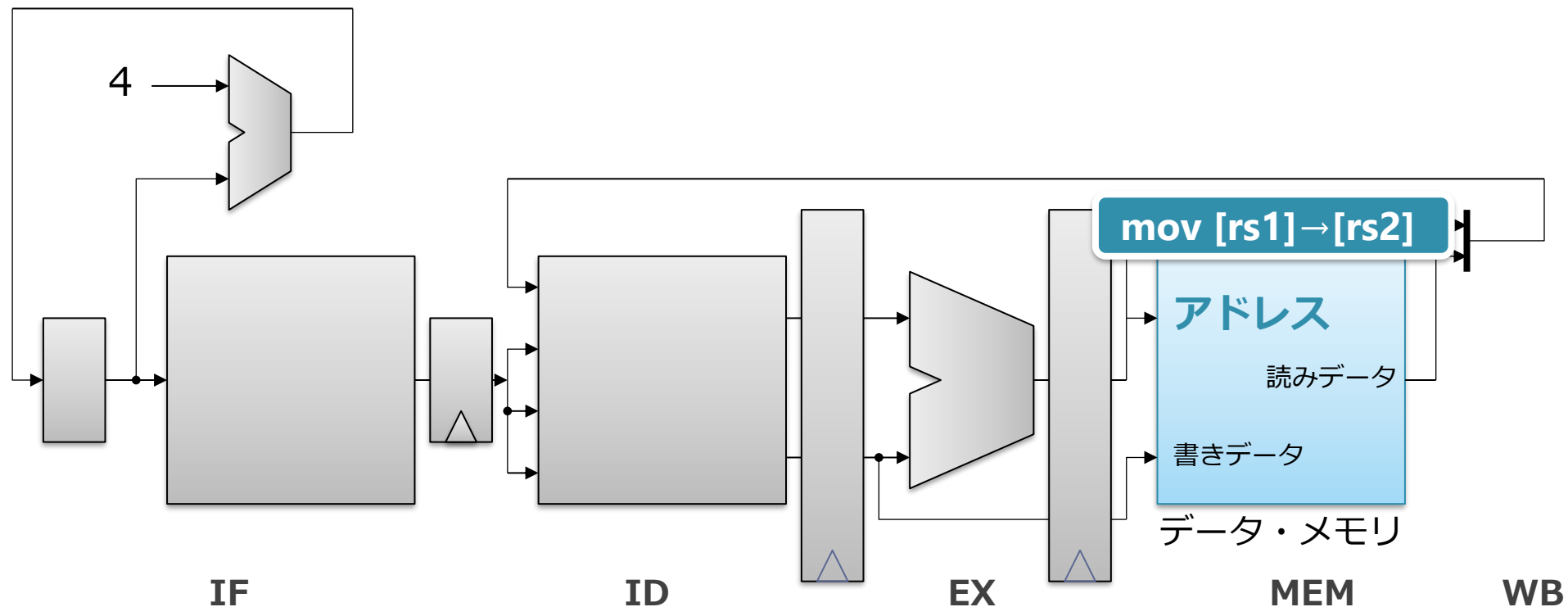
■ 構造ハザード：

- ハード資源の不足により, パイプラインがうまく動作しないこと
- いくつかの例を使った説明, 解消方法について解説

構造ハザードの例 1 : メモリ間 mov

- 例 1 : 仮に `mov [rs1]→[rs2]` のような命令があったとする
 - メモリを読んで、その結果をメモリに書く (=コピー)
- 具体的な動作：
 - `rs1` で指定されるアドレスのメモリの値を読んで、
 - `rs2` で指定されるアドレスのメモリに書き込む
- 実際に、x86 にはこのような命令がある

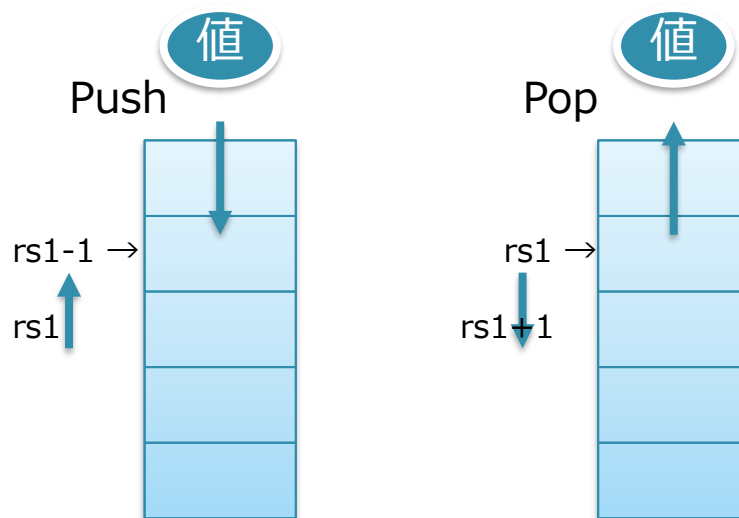
mov [rs1]→[rs2] // [rs1]→[rs2] へのコピー



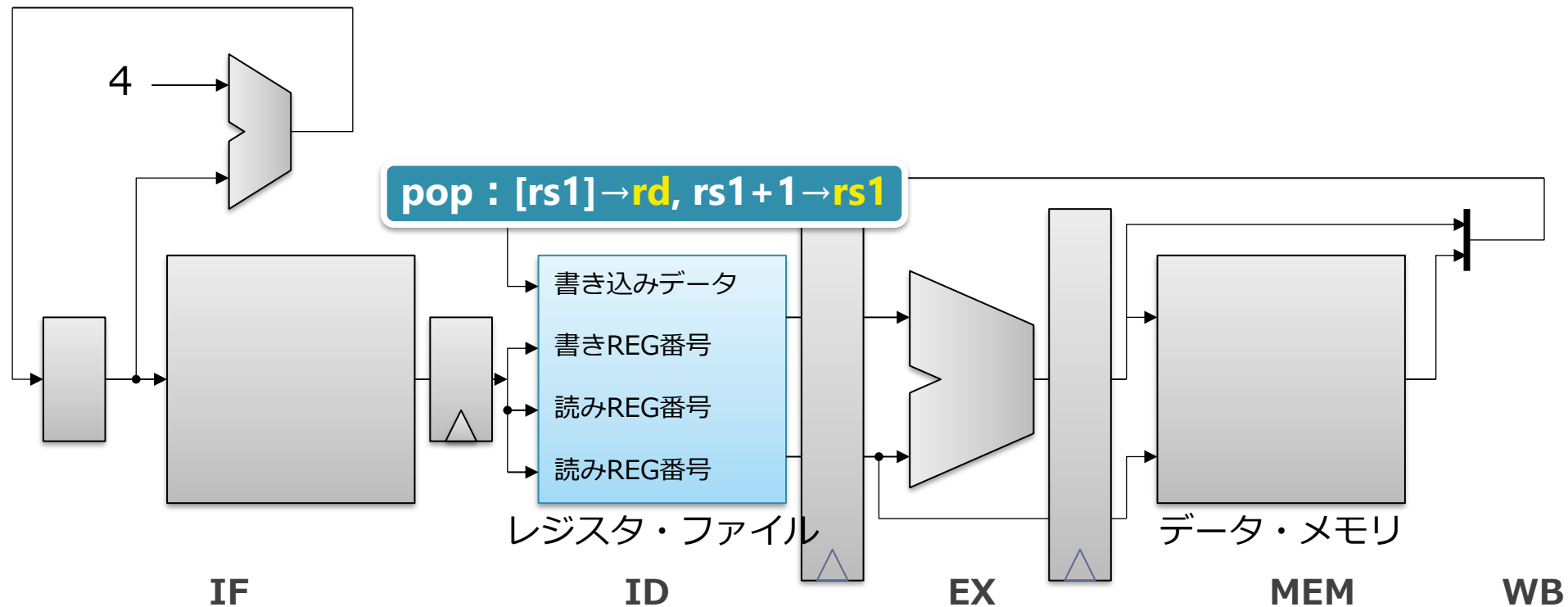
- ある1つのサイクルにメモリを「読んで」「書く」必要がある
 - しかし、データ・メモリのアドレスの口は1つしかない
 - MEM ステージでデータ・メモリの読みと書きが同時にできない

構造ハザードの例 2 : push/pop

- x86 や ARM ではスタック操作のための push/pop 命令がある
 - push : $rs1-1 \rightarrow rd$, $r2 \rightarrow [rd]$
 1. スタック・ポインタ（が入ってるレジスタ）をデクリメントし,
 2. それをアドレスにしてメモリに値を書き込む
 - pop : $[rs1] \rightarrow rd$, $rs1+1 \rightarrow rs1$
 1. スタック・ポインタをアドレスにして値を読む
 2. スタック・ポインタをインクリメント



pop : [rs1]→rd, rs1+1→rs1



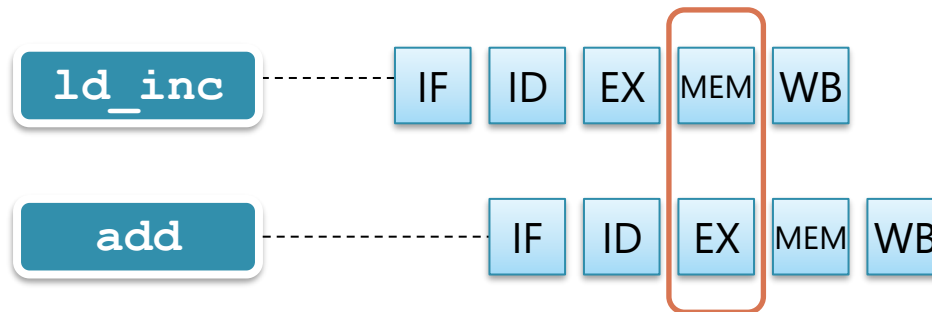
- WB ステージでレジスタに `rd` と `rs1` の2つを書き込む必要がある
 - 2つのレジスタが1つの命令により更新されている
 - レジスタ・ファイルへの書き込みは、同時に2つはできない

構造ハザードの例 3

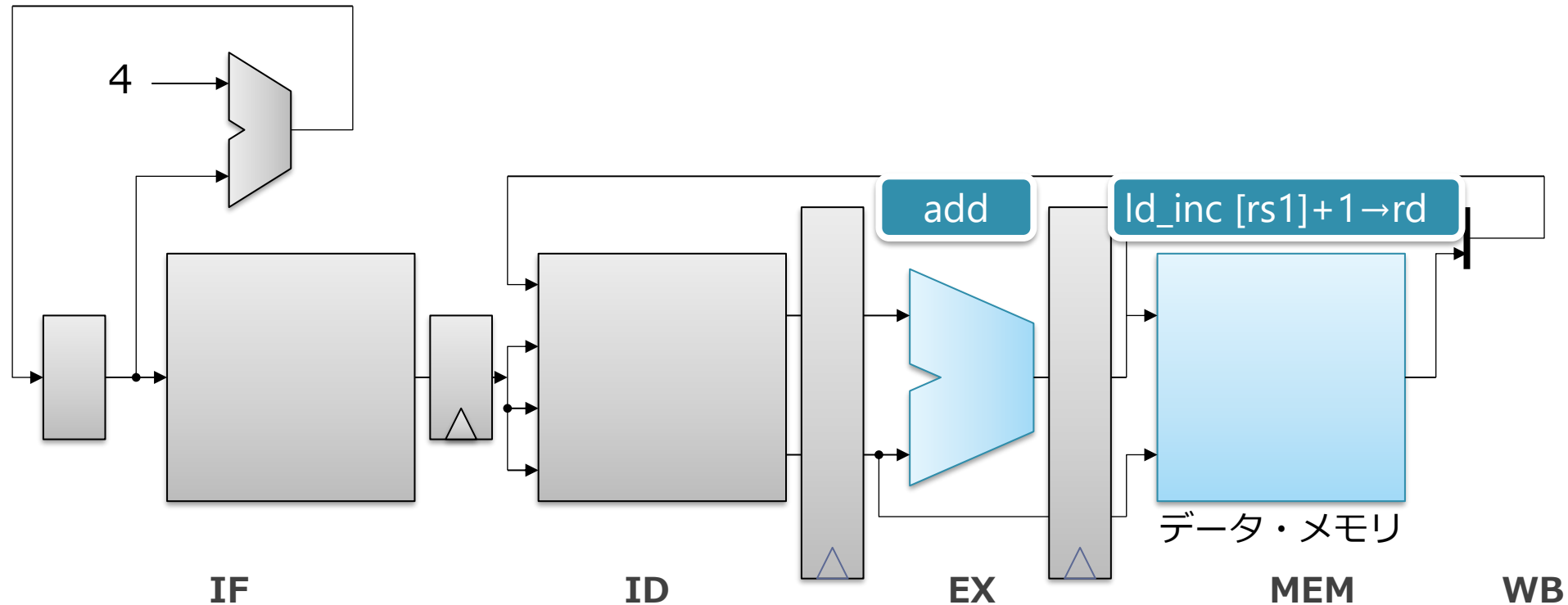
- 使用資源の異なるステージ間のぶつかりでも起きる
 - これまでの例は, 同じステージ内で資源が足りない例
- `ld_inc [rs1]+1→rd` のような命令があったとする
 1. `rs1` の指すアドレスからメモリを読む
 2. 読んだ値にさらに + 1 してから `rd` に書く
- 一見, 資源は足りているようだが…
 - レジスタの読み書きは 1 つずつしかない
 - メモリも 1 カ所を読むだけ
 - 加算も 1 回行っただけ

構造ハザードの例 3

- `ld_inc [rs1]+1→rd` と `add` が連続した場合：
 - `ld_inc` で, MEM ステージから読んだ値を加算しようとしても,
 - そのサイクルは後続の `add` が演算器を使っているので使用できない



ld_inc [rs1]+1→rd と add が連続した場合



- 命令は EX ステージ以外では、演算器にはアクセスできない
 - そうしないと、他の命令の EX ステージとかち合ってしまう

構造ハザードの解決方法

■ 解決方法

1. ハードウェアの増強
2. 時分割処理

解決方法 1 : ハードウェアの増強

■ ハードウェアを増強する

- `mov [rs1]→[rs2]`
 - 複数箇所のメモリを同時に読み書きできるように
- `pop`
 - レジスタに2つ同時に書き込めるように
- `ld_inc [rs1]+1→rd`
 - MEM ステージに専用の加算器を追加

解決方法 1 : ハードウェアの増強

- 利点 : オーバーヘッドをいとわなければ, 基本これで解決
- 欠点 : 回路規模が増える
 - 1. 機能の増強量に比例した回路が必要
 - なにも考えないで対応していくと, ものすごい数の回路になる
 - 例 : ARM は全 16 レジスタを一気にメモリに書ける命令がある
 - 2. 機能の増強量に対して, 線形より大きなオーダーで回路規模が増える場合もある
 - 加算器などなら, 増やした数の分だけ線形に回路が増える
 - メモリやレジスタは, 同時に読み書きできる数の2乗で回路が大きくなる性質がある

構造ハザードの解決方法

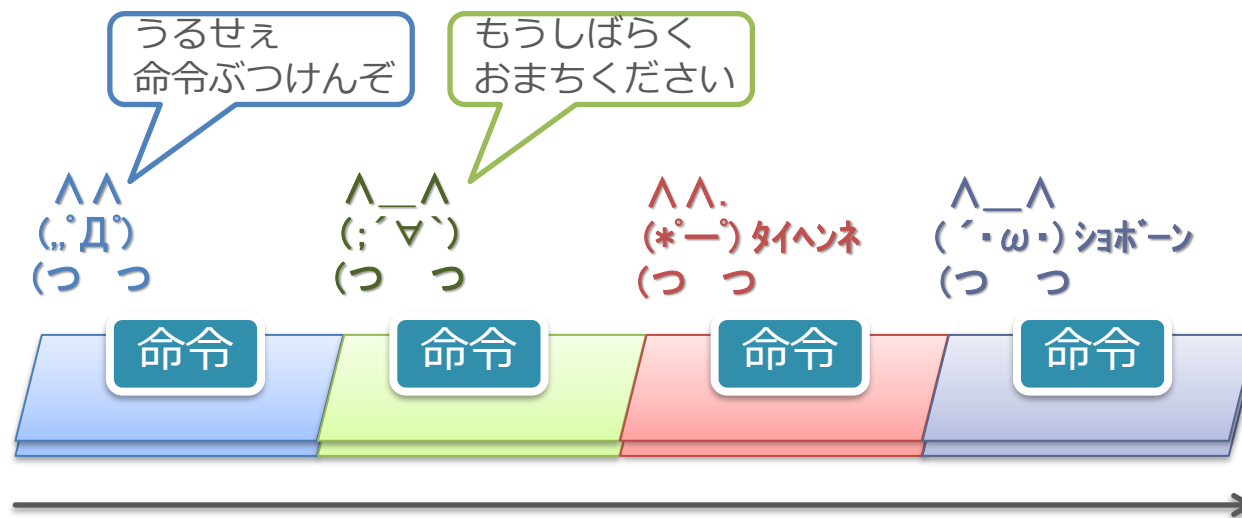
■ 解決方法

1. ハードウェアの増強
2. **時分割処理**

解決方法 2 : 時分割で処理

- 構造ハザードの原因：
 - ハードウェア（の機能）が足りない
- **パイプラインを止めて**，複数のサイクルをかけて処理する
 - `mov [rs1]→[rs2]`
 - メモリを読んだあと，次のサイクルで書きこむ
 - `pop`
 - 1つレジスタに書いたあと，次のサイクルで書き込む
 - `ld_inc [rs1]+1→rd`
 - `ld_inc` が MEM で値を読んだら，次のサイクルで +1

なぜパイプラインを止めるのか

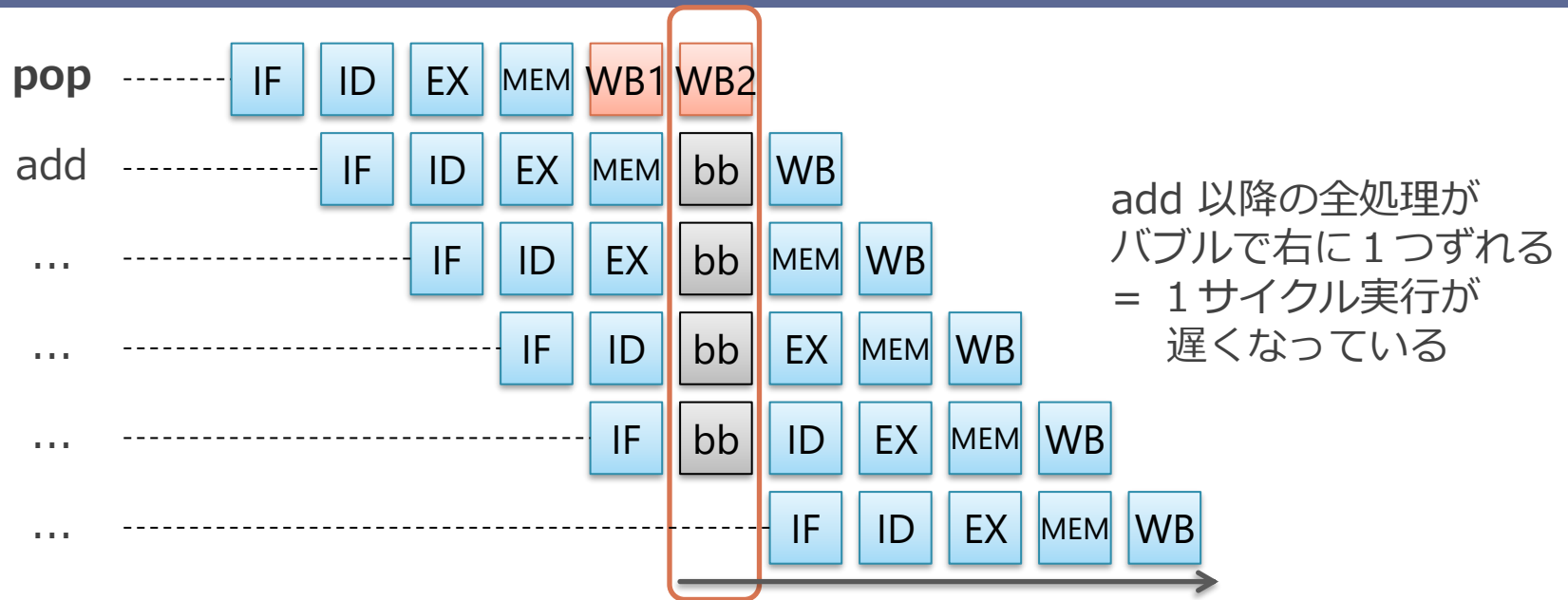


- 上流を止めないと破綻する
 - ($;\text{'}\nabla\text{'}$) が複数サイクルをかけて仕事をしている場合、命令はそこにとどまり続ける
 - その間は上流をとめないと命令をおく場所がないし、依存関係がまもられない
- ($*-\text{'}$) より下流は流れていっても、この場合は問題ない

パイプラインを止めること

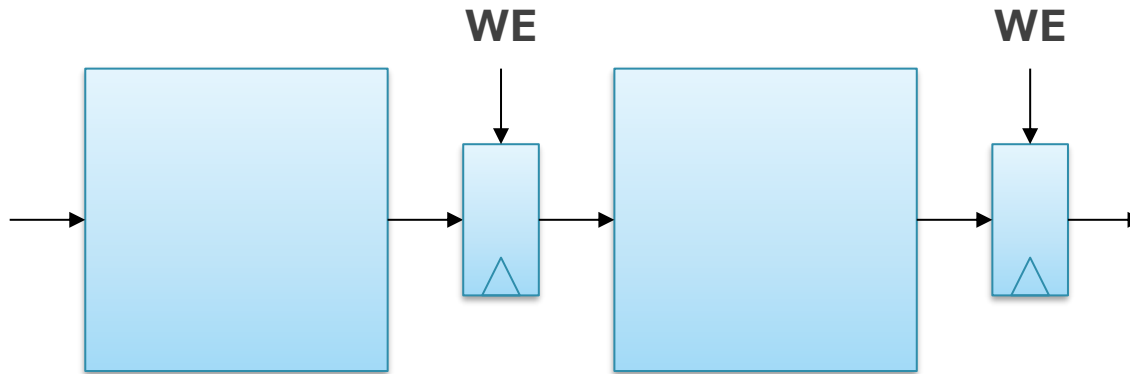
- パイプラインを止めるとことを「ストール」や「インターロック」という
 - 本や人によって、意味や使い方が微妙に統一されていない
 - この講義では、以降はストールで統一

ストールの動作



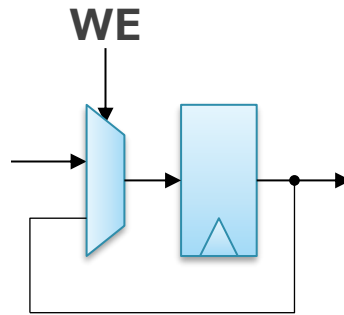
- `pop` : 1つレジスタに書いたあと、次のサイクルで書き込む
 - WB1 と WB2 の2サイクルで書き込む
 - WB2の間は上流を全て止める
- パイプライン・チャート上では上記のようになる
 - 止める原因の命令の下が全部右にずれる
 - ずれた部分の空き (bb) を「バブル (bubble)」とよぶ

ストールの実現方法



- 回路的には、Write Enable (WE) つきの D-FF を使う
 - WE が 0 のサイクルは書き込みが行われない
 - ストールさせたい時は、そのステージの WE を 0 に

WE つき D-FF の実現方法



- たとえば D-FF とマルチプレクサで作れる
 - WE が 1 の時は, 左からきた入力を選んで書き込む
 - WE が 0 の時は, その時の自分自身の出力を選んで書き込む

非構造ハザード

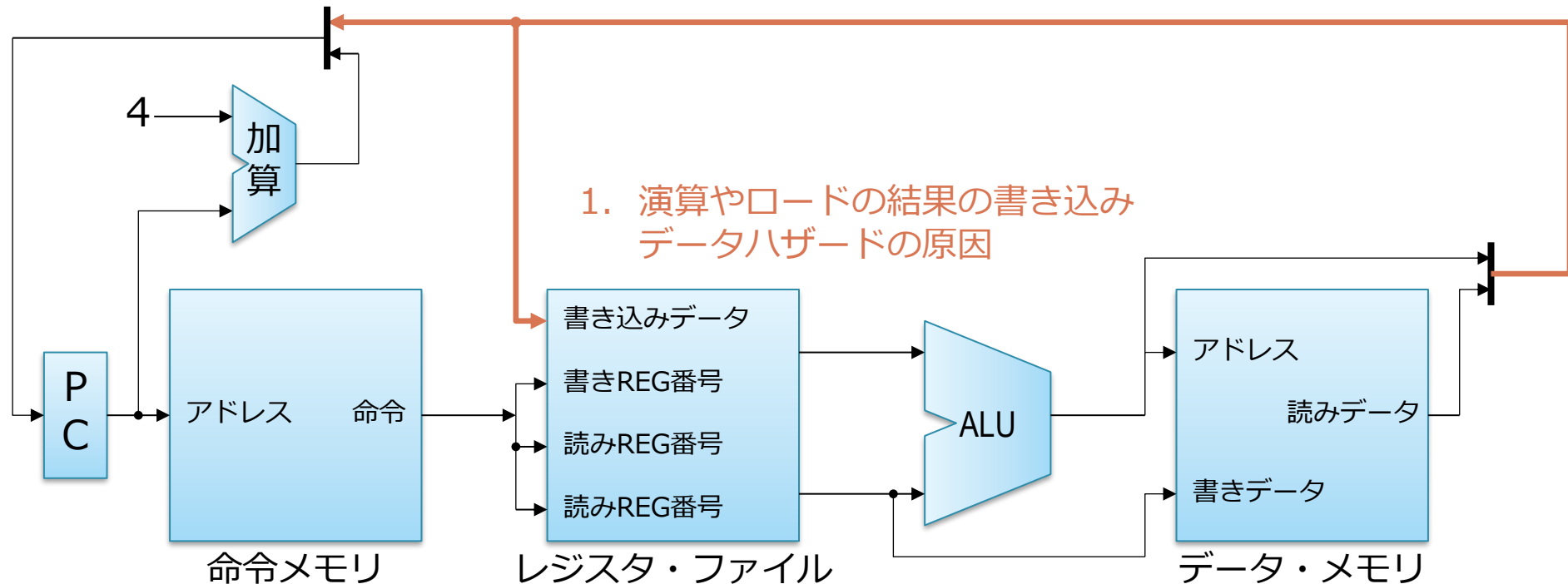
もくじ

1. 構造ハザード：ハード資源の不足に起因
 1. 構造ハザードとはなにか？
 2. その解決方法
2. **非構造ハザード：バックエッジに由来**
 - a. データ・ハザード
 - b. 制御ハザード

バックエッジとは：逆方向（右から左）にいく信号

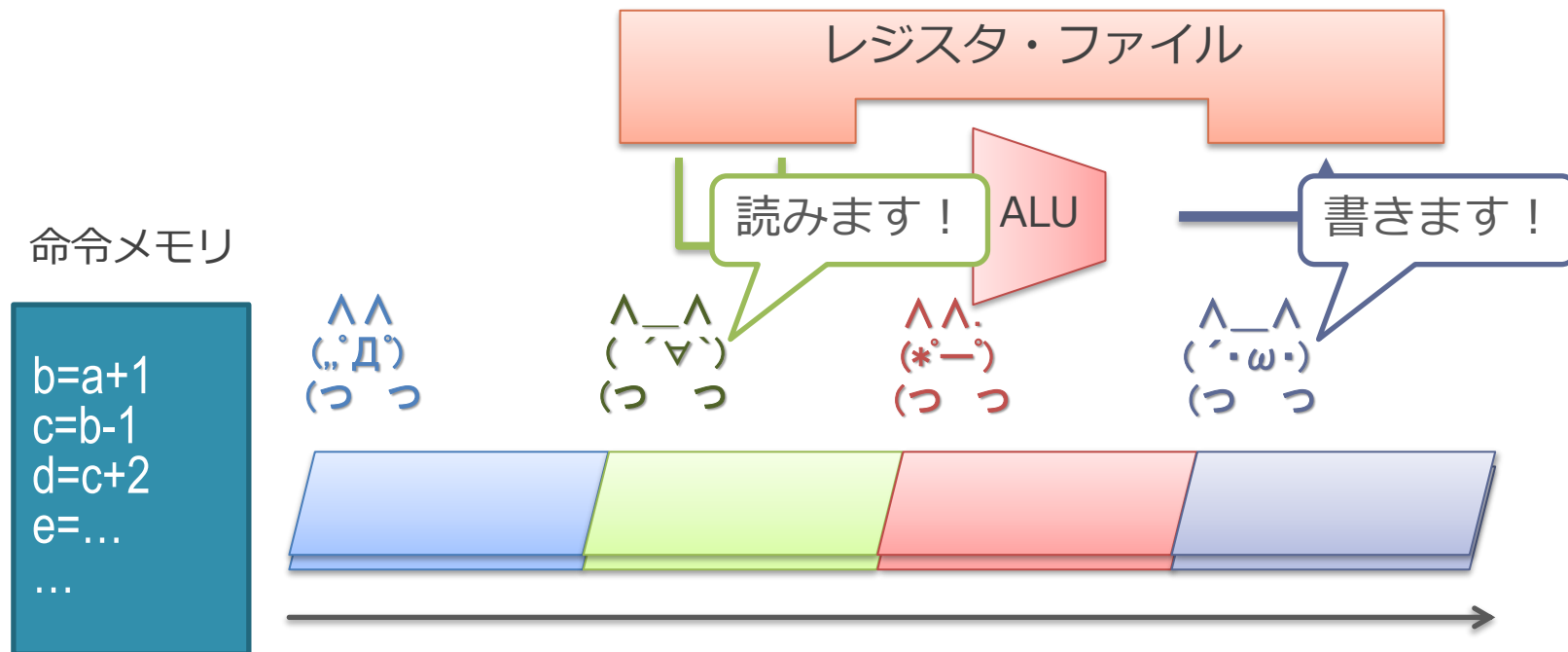
2. 分岐結果の PC への反映
制御ハザードの原因

1. 演算やロードの結果の書き込み
データハザードの原因



- バックエッジがあるため、命令を単純に流せない場合がある
 - 工場のラインのように、一方向に流せない

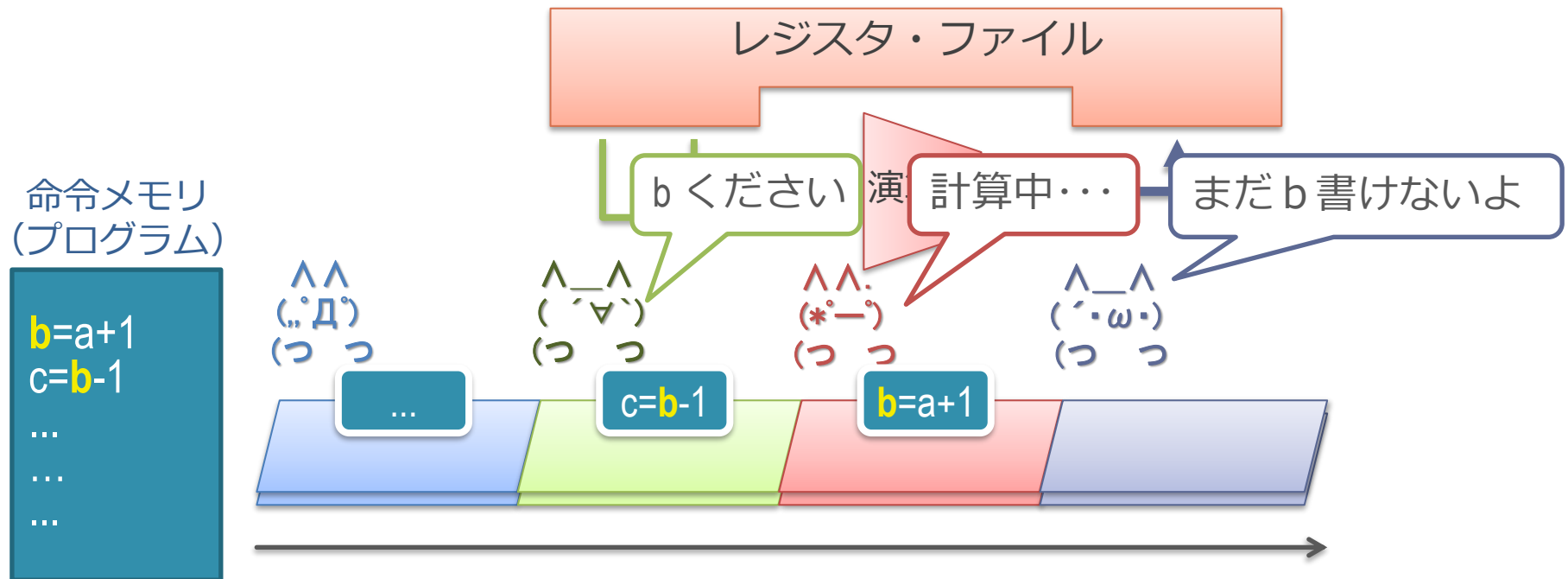
データ・ハザード



■ レジスタ・ファイルへのアクセス

- 演算の入力は(´▽`)の人がレジスタ・ファイルから読み出す
- 演算の結果は(´・ω・`)の人がレジスタ・ファイルに書き込む

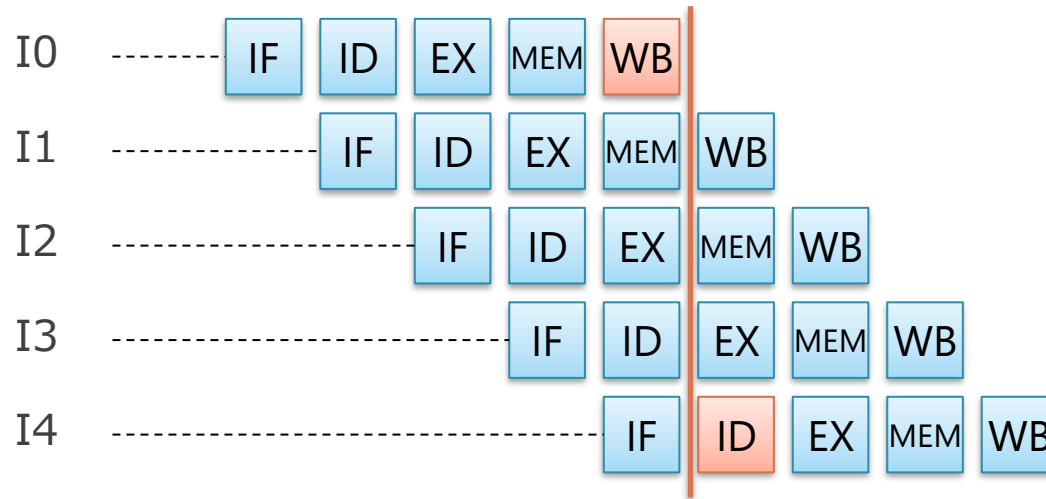
データ・ハザード



■ 直前の命令の結果を使う命令が現れた場合：

- (' ▽ `) の人が $b=a+1$ の結果を読もうとしても,
- (* ° —) の人がまだ計算中でレジスタ・ファイルに b が書けていない
- (` · ω ·) の人が計算結果をかけるのはさらに次のサイクル

データ・ハザード



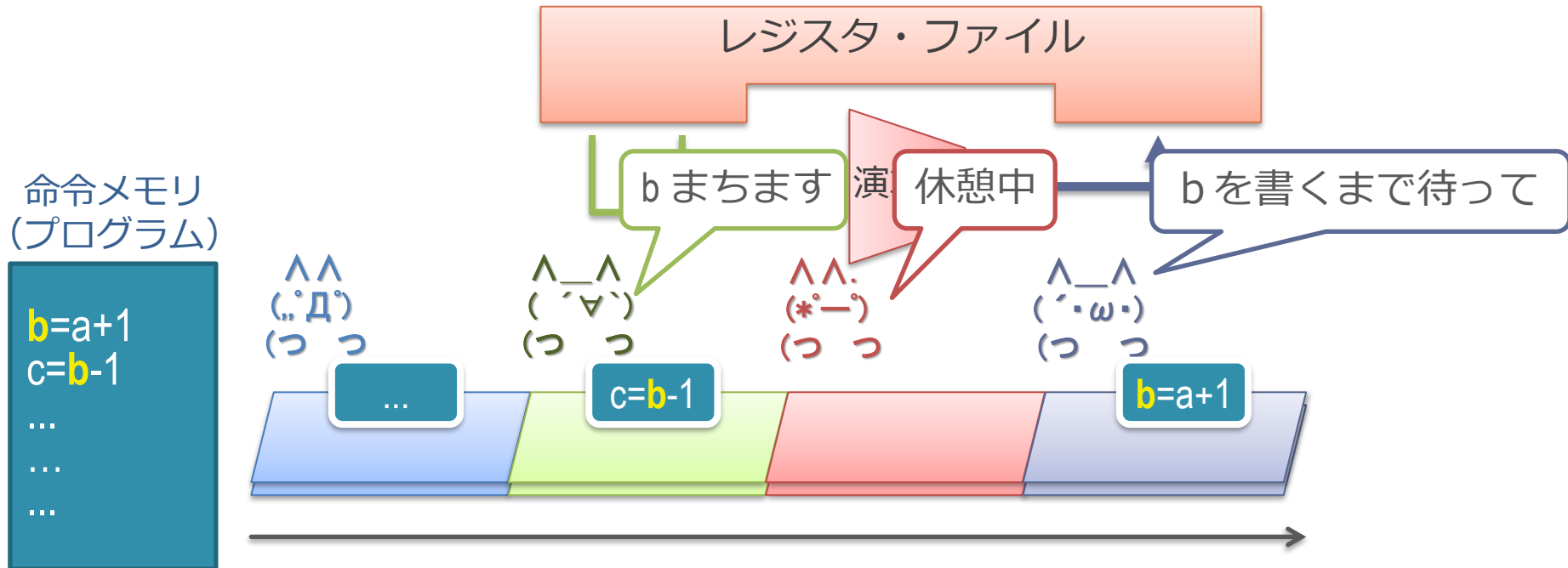
- I0 の WB が終わるまで、その結果はレジスタに書き込まれない
 - I4 までは、その値がレジスタから得られない
 - ID ステージでレジスタを読むため

データ・ハザードの解消方法

■ 解消方法

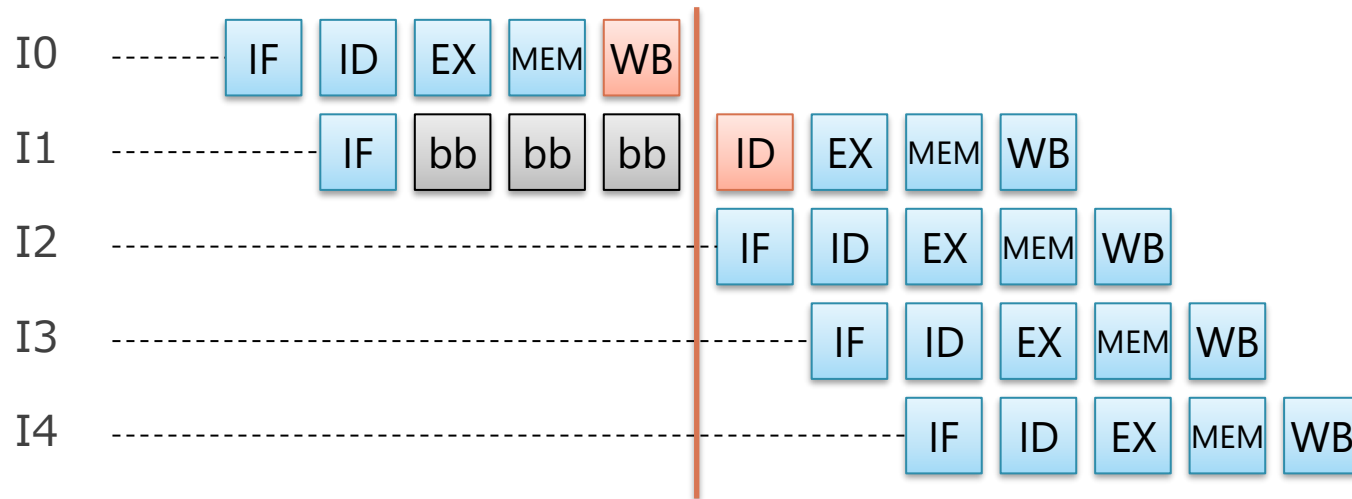
1. ストールさせる
2. 遅延スロット（なにもしない）
3. フォワーディング

1. ストール



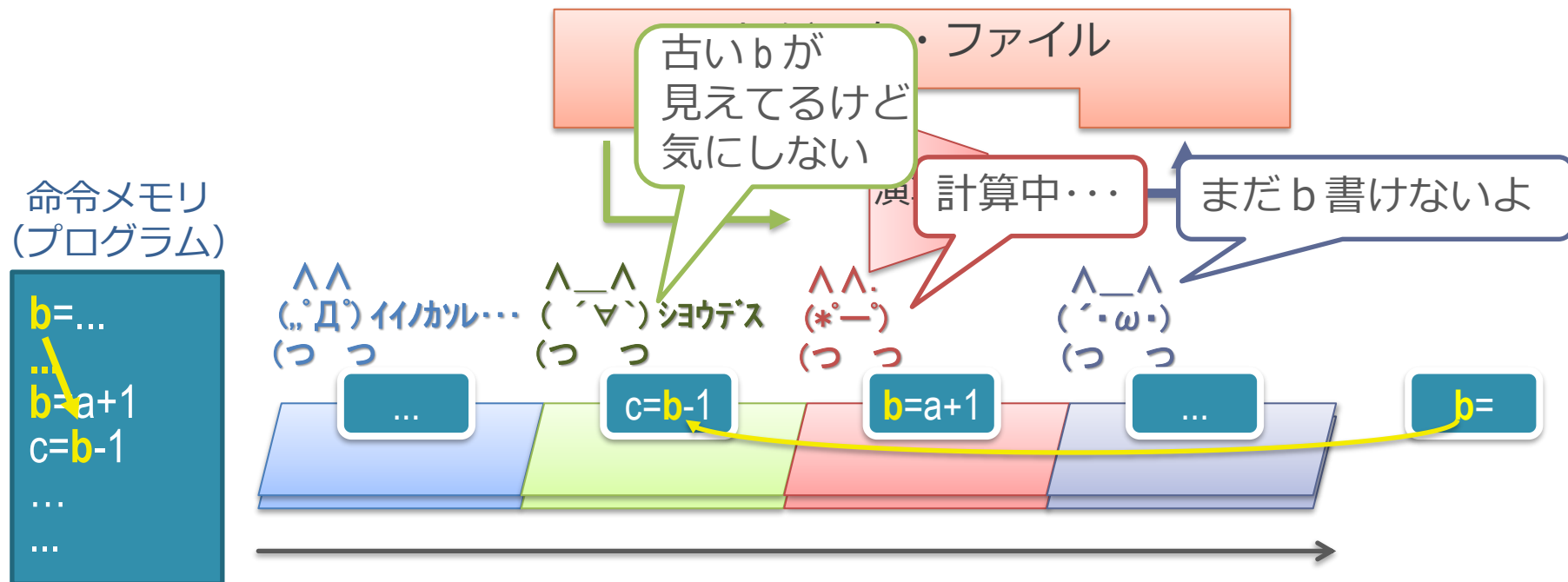
- 直前の命令の結果を使う命令が現れた場合：
 - $(\cdot \omega \cdot)$ の人が計算結果をレジスタに書くまで,
 $(\cdot \nabla \cdot)$ の人と上流のラインを止める
 - 間にバブル (何もしないステージ) が入る

1. ストールさせる



- I0 の WB が終わるまで、後続の命令を遅らせる
 - I1 の ID が、I0 の WB の右にくるまでストール
 - I1 は I0 の結果を使える
- 欠点：プログラムの実行がとても遅くなる

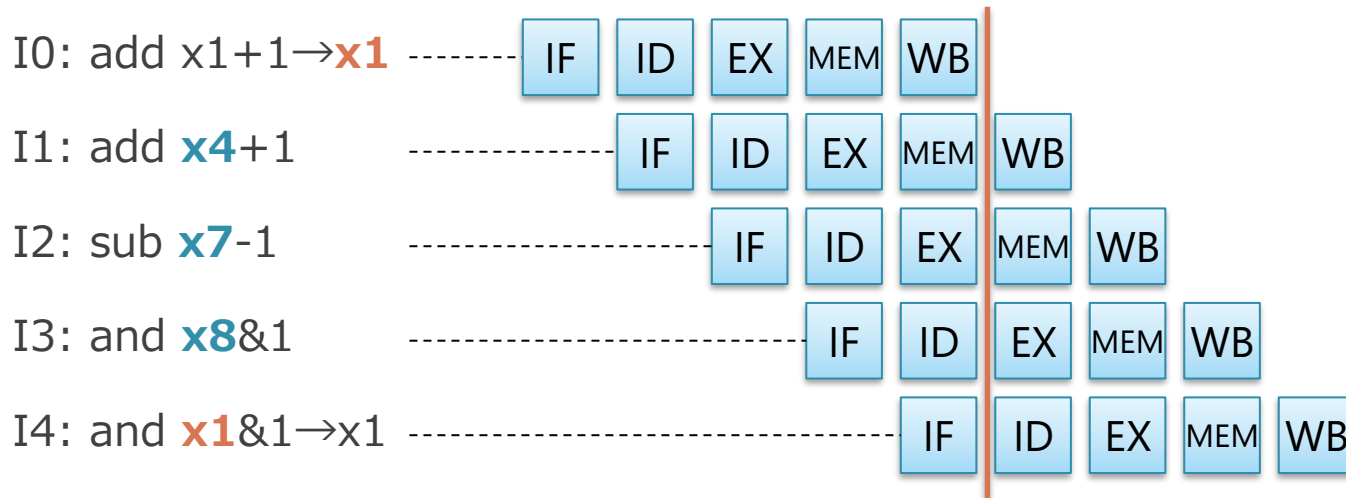
2. 遅延スロット（なにもしない）



■ 直前の命令の結果を使う命令が現れた場合：

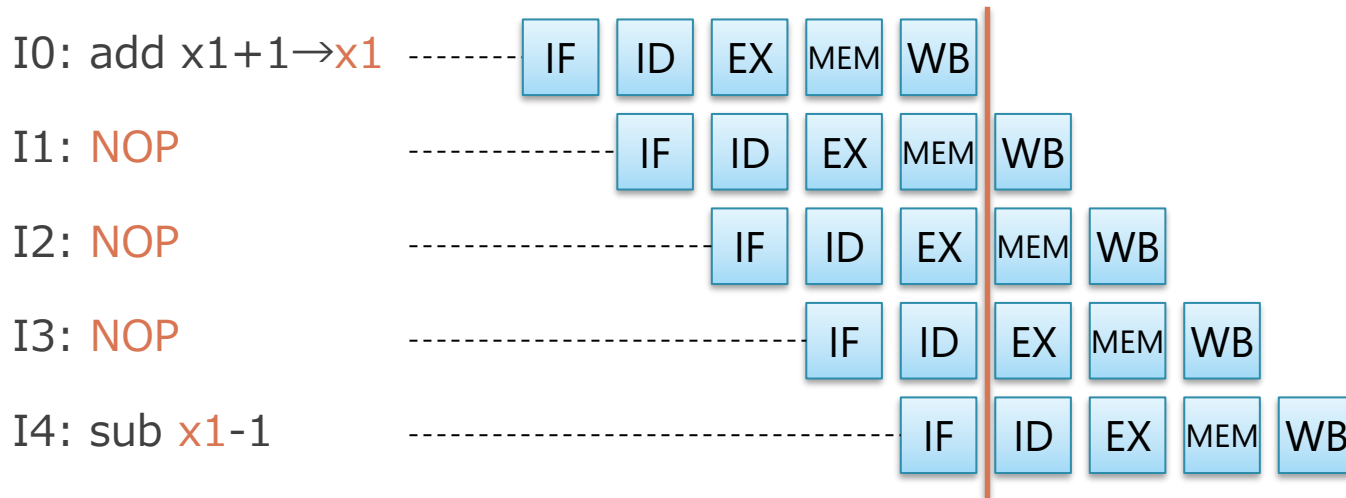
- ('▽') の人は、その時レジスタ・ファイルにある b を気にせず読んでしまう
- 2つ前の命令の結果の b が読めてしまう
 - そういう仕様ということにする

2. 遅延スロット（なにもしない）



- ここに I0 の結果を使わない命令を入れれば、性能低下はない
 - この直前の命令の結果が見えない部分を「遅延スロット」と呼ぶ
 - この図では、遅延スロットが 3 命令分ある
 - それらは I1, I2, I3 は I0 の結果を使っていないので問題無い
 - 遅延スロットへの命令挿入はコンパイラががんばる
 - 人力でアセンブリ言語でがんばることもある

NOP の挿入



- もしそのような命令がない場合,
 - NOP (No Operation) と呼ぶ何もしない命令をいれる
 - これもコンパイル時にいれておく必要がある
- 上の例は, x1 に 1 を足した結果を使う以外の処理がなかった場合

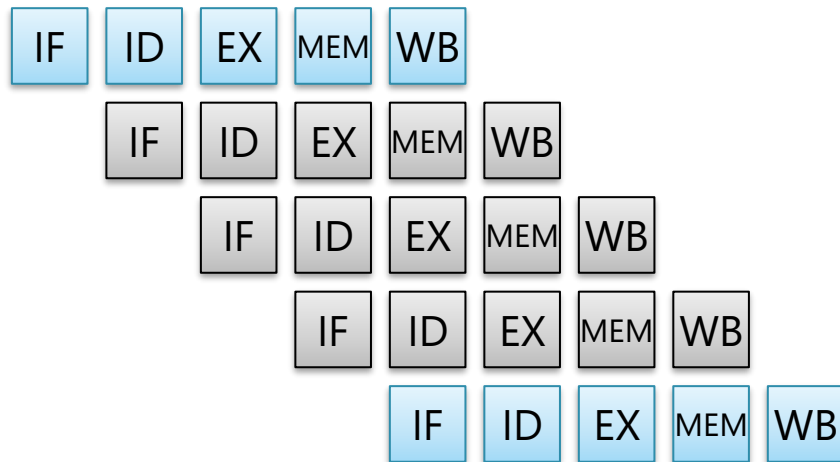
遅延スロットの利点

■ 利点：

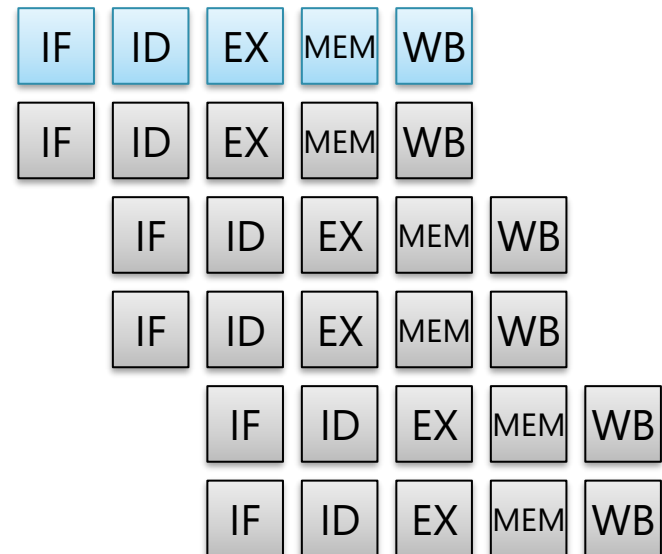
- なにもしないので，ハードは最も単純
- 並列にできる命令があれば，性能も下がらない

遅延スロットの欠点

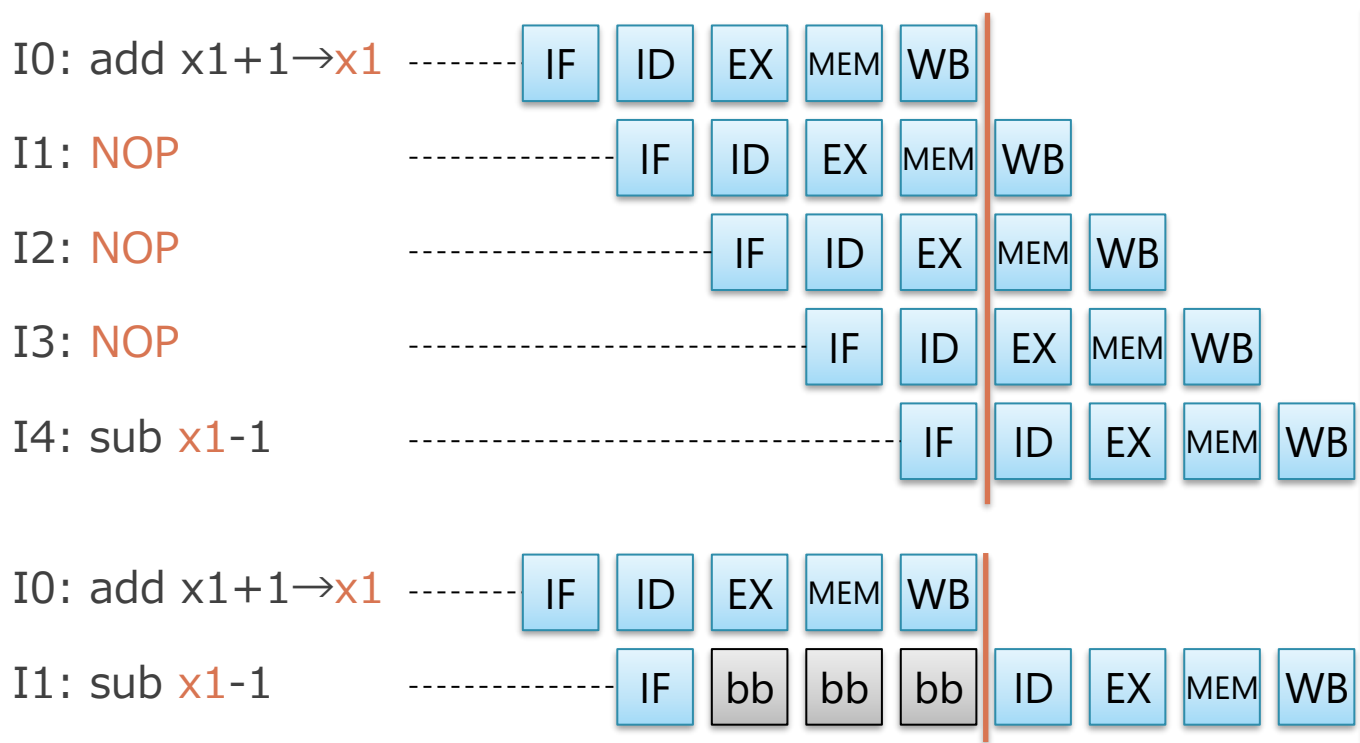
- 欠点：「仕様」なので，一度決めると変えられない
 - 後からパイプラインの段数や構造を変えると互換性がなくなる
 - クロックをあげるために，段数を増やせない
 - 複数の命令を同時処理しようとしたときにも互換性がなくなる
 - MIPS では遅延スロットが 1 命令分，仕様として存在
 - 互換性のためにこれを忠実に再現するため後年は逆に複雑化



2 命令同時処理すると，遅延スロットが増える



遅延スロットの欠点2



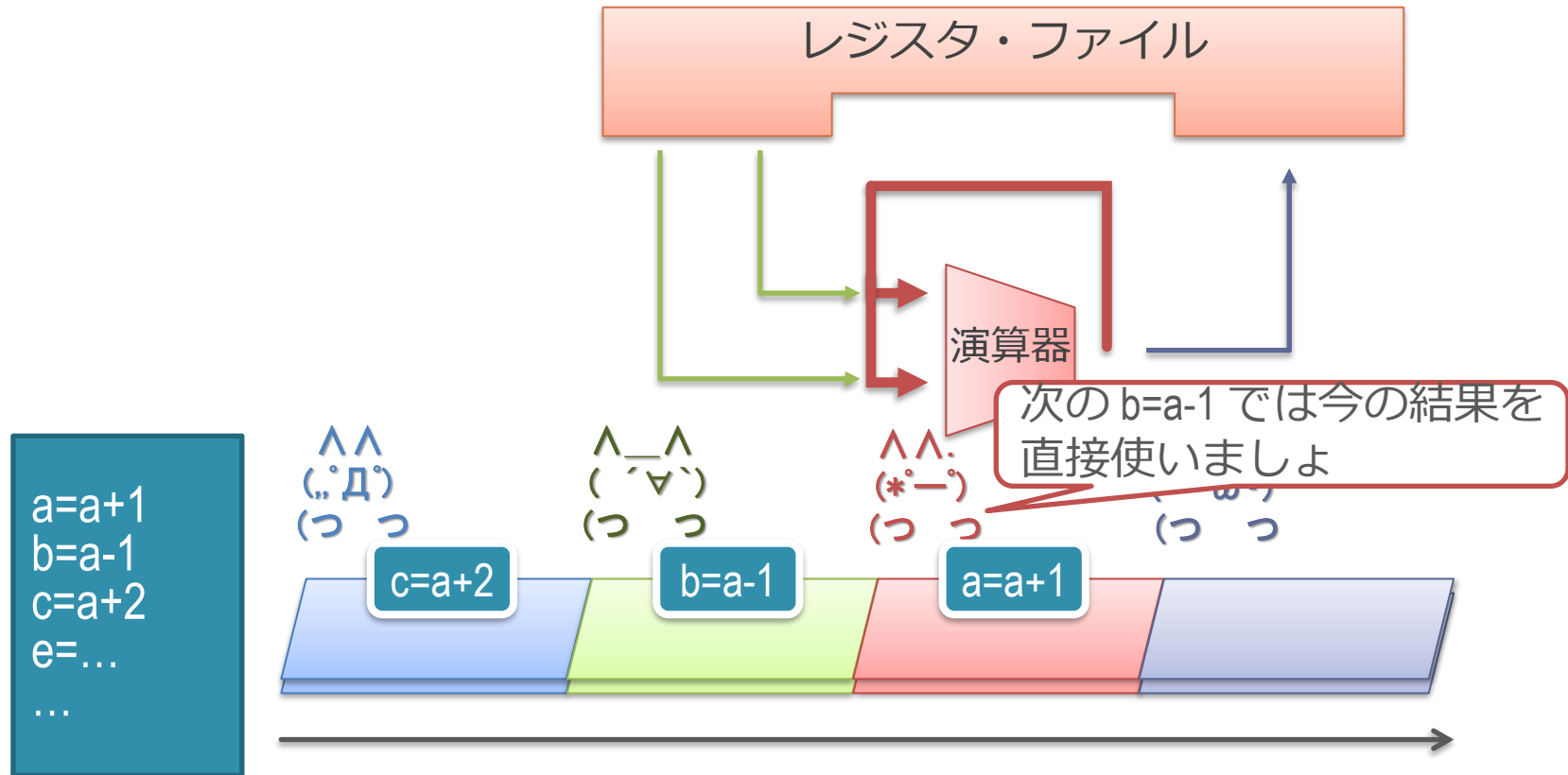
- 欠点2：並列してできる命令が常にあるとは限らない
 - NOPを入れるしかなくなる
 - 実質ストールしてバブルを入れるのと同じになってしまう

データ・ハザードの解消方法

■ 解消方法

1. ストールさせる
2. 遅延スロット（なにもしない）
3. **フォワーディング**

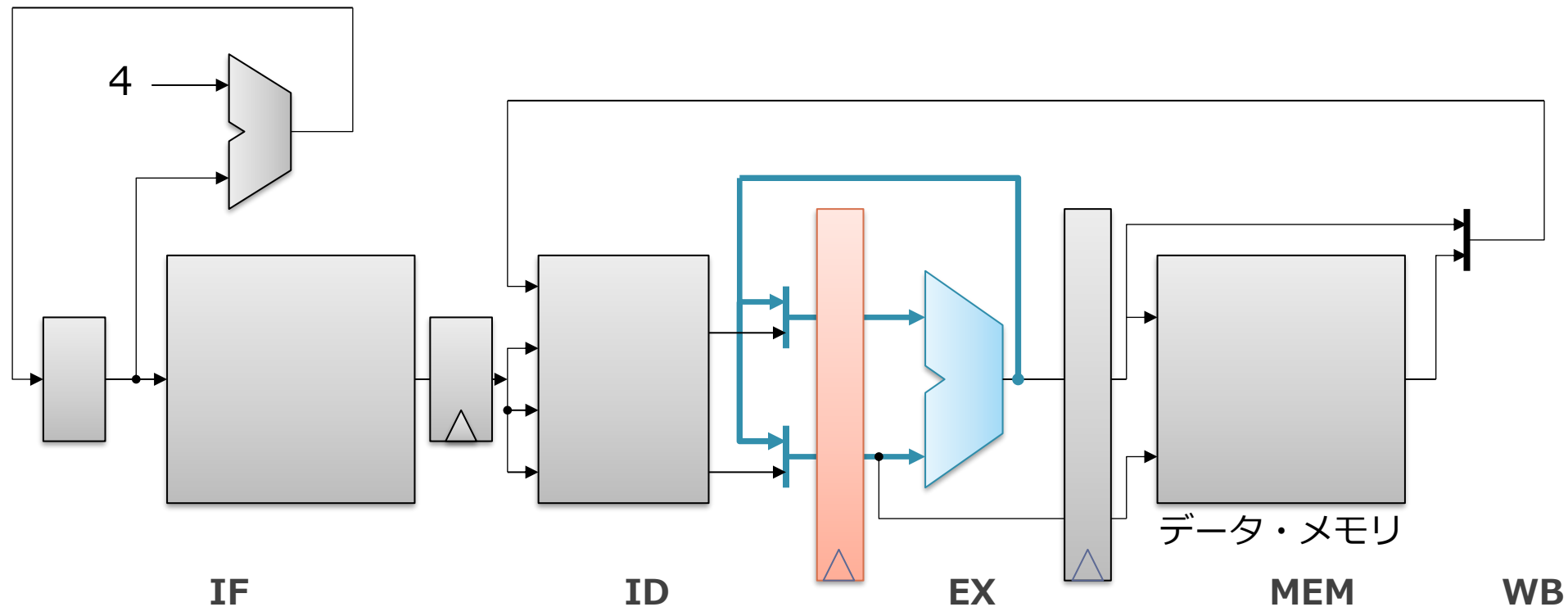
フォワーディング



■ フォワーディング (バイパスとも呼ぶ)

- $(\ast -)$ の人が、次のサイクルに自分の計算結果を即座に使えるよう、手元にも結果を置いて使う
- $(\wedge \nabla)$ がレジスタ・ファイルから読んできた値は必要に応じて捨てる

フォワーディングの回路



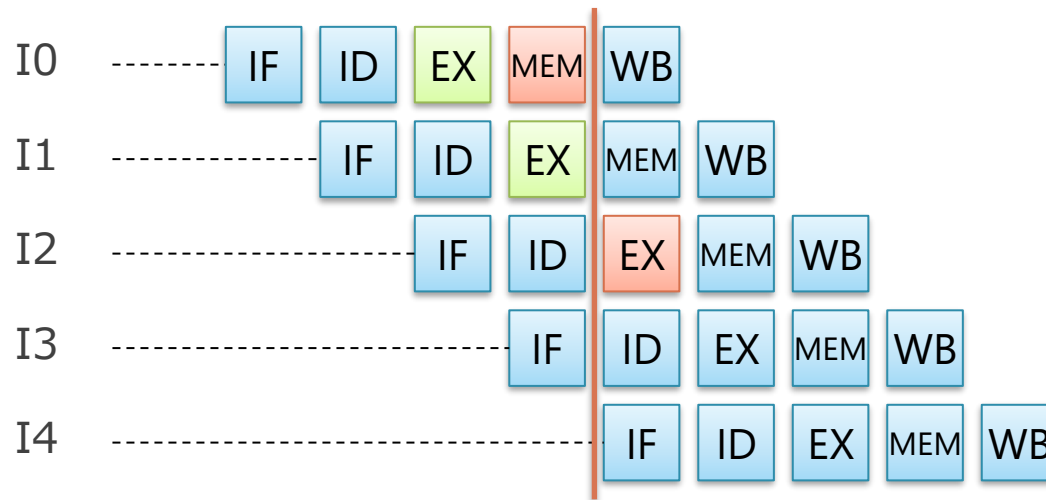
- 演算器の結果を, フィードバック
 - レジスタ・ファイルからの読み出し結果と選択して入力に

フォワーディングの利点

■ 利点：

- 依存関係がある命令が連続できてもパイプラインを動かし続けられる
- バブルを発生させることがない

ロードについては、完全に解決はできない



- ロードではデータ・メモリを読むまでその値は取れない
 - 次の命令は、MEM より後に EX がこないといけない
 - I1 は、I0 のロード結果が見えない
 - この部分はストールや遅延スロットでなんとかすることがおおい

制御ハザード

1. 構造ハザード
2. 非構造ハザード：バックエッジに由来
 - a. データ・ハザード
 - b. 制御ハザード**

分岐命令の処理と制御ハザード



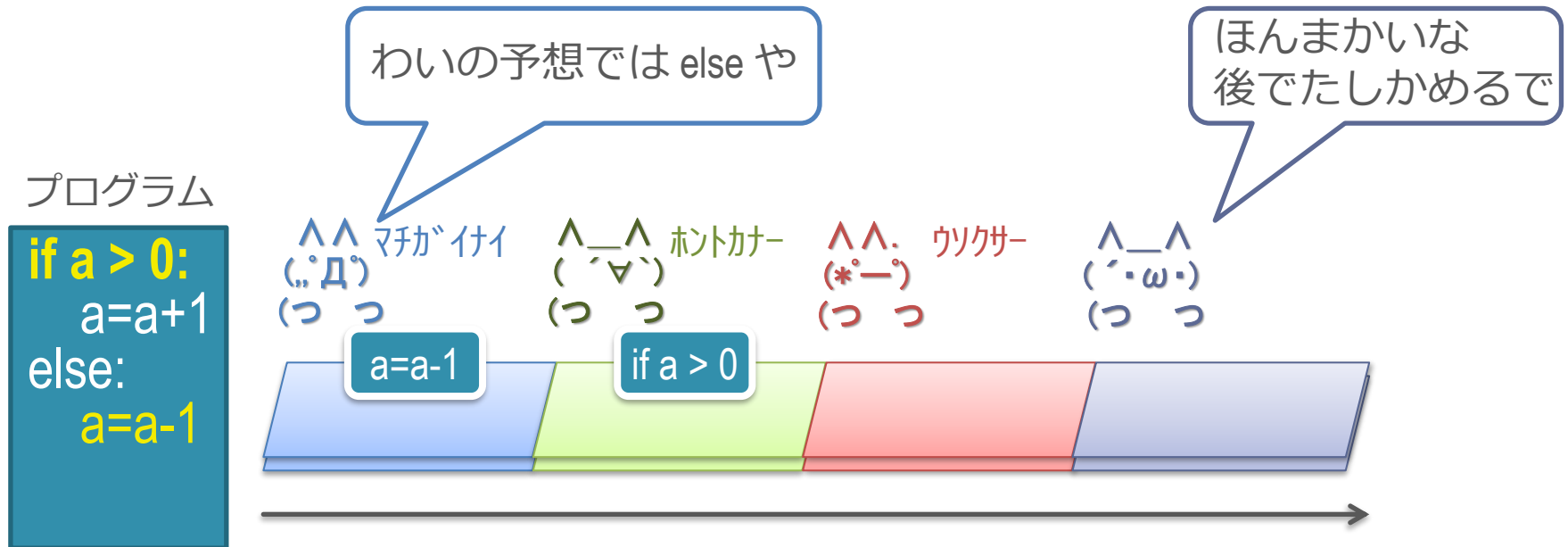
- 「if a > 0」の結果は最終段の(´・ω・)の人まで反映出来ない
 - 先頭は次に a=a+1 と a=a-1 のどちらを取り込めばいいのかわからない

制御ハザードの解消方法

■ 解消方法

1. ストールさせる
 2. 遅延スロット（なにもしない）
- 上記は基本的にデータ・ハザードと同様にして適用できる
 - ただしフォワーディングは、制御ハザードでは意味的に無理
3. 分岐予測による投機実行

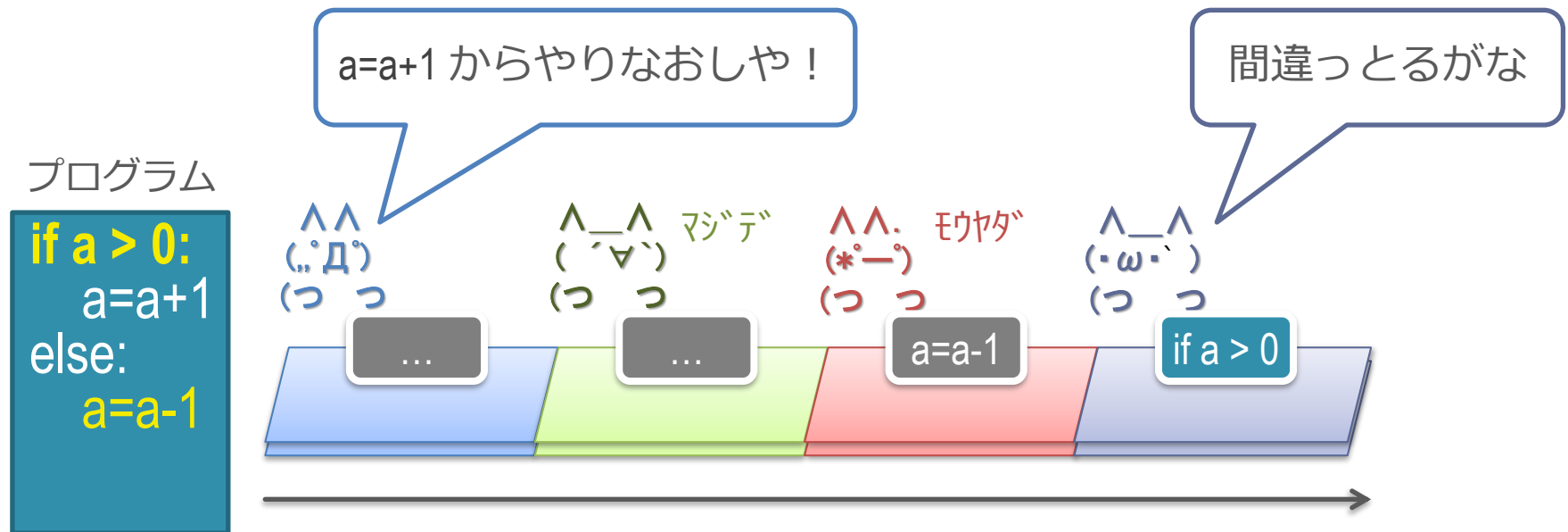
分岐予測



■ 動作

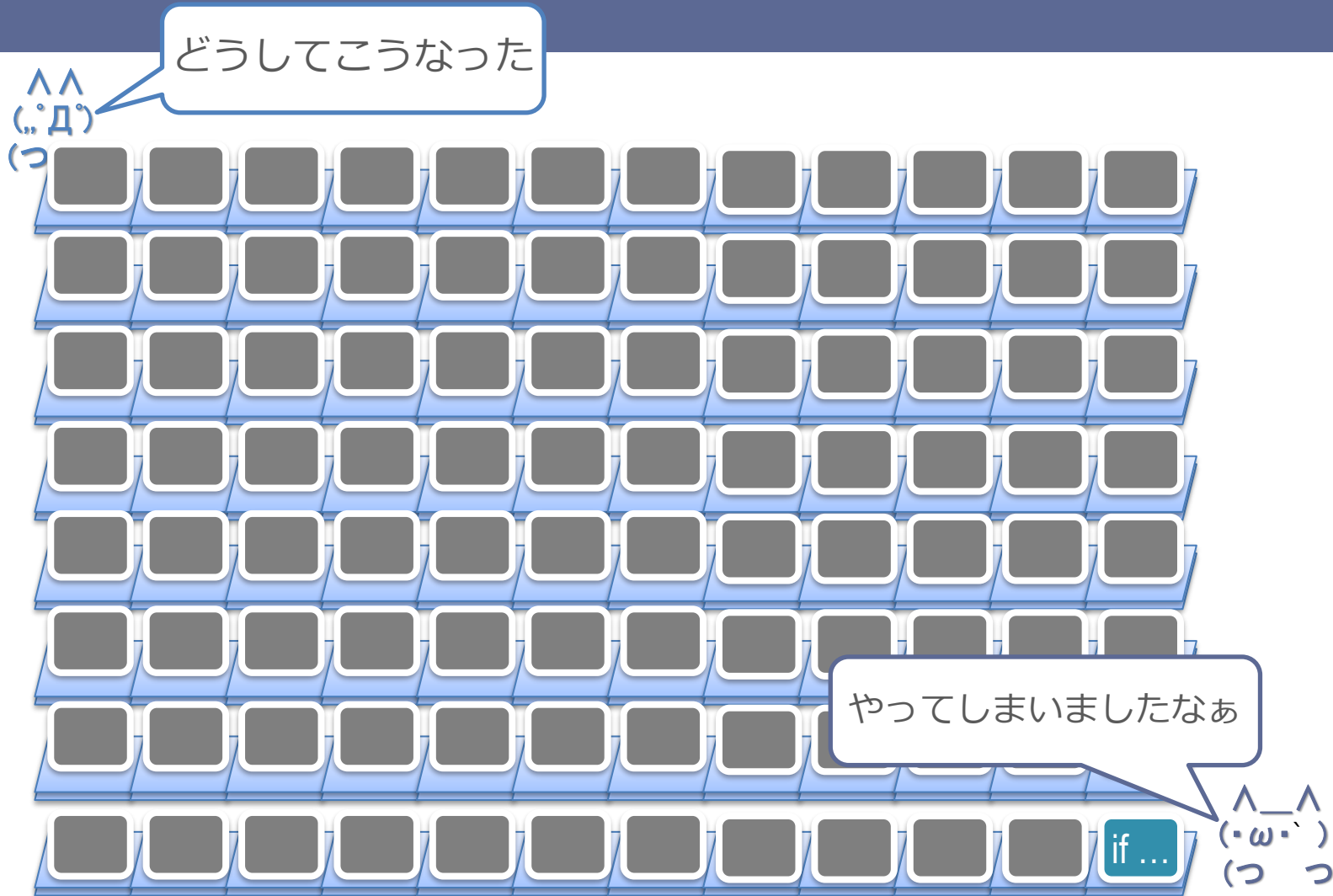
- 「if a > 0」の結果を予測して、命令を取り込む
 - 前はこっちに行ったので、次もこっちに違いないとかで予測
- あとから予測が正しいか確認する

分岐予測ペナルティ



- 予測が間違っていた場合，以降の処理を取り消してやり直す
- この図では，無駄になるのは3命令分

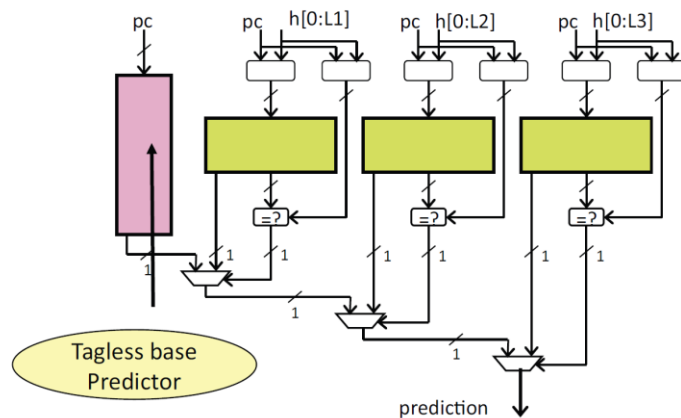
大規模な高性能プロセッサの場合



■ 取り消しは最悪数十命令以上に

- Apple の M4P という CPU だと 10命令同時 × 10数段

分岐予測のアルゴリズム



■ André Seznec 先生が作った TAGE という予測器が現在最も予測精度が高いと言われている

- A. Seznec and P. Michaud. A case for (partially)-tagged geometric history length predictors, *Journal of Instruction Level Parallelism* (<http://www.jilp.org/vol8>), 2006
- 写真は <https://team.inria.fr/pacap/members/andre-seznec/> より

■ Championship Branch Prediction という大会がある

- 分岐予測は、この大会で出た様々なアイデアを元に発展してきた
- Seznec 先生がもうずっと優勝し続けてる
- 今年は久々に開催されるので、塩谷も挑戦

パイプライン化の限界

- 速度が上がらなくなる理由：

1. 回路的な理由による周波数向上の限界（前回の講義）
2. **アーキテクチャ的な理由（ハザード）による実効性能の限界**

アーキテクチャ的な理由による実効性能の限界

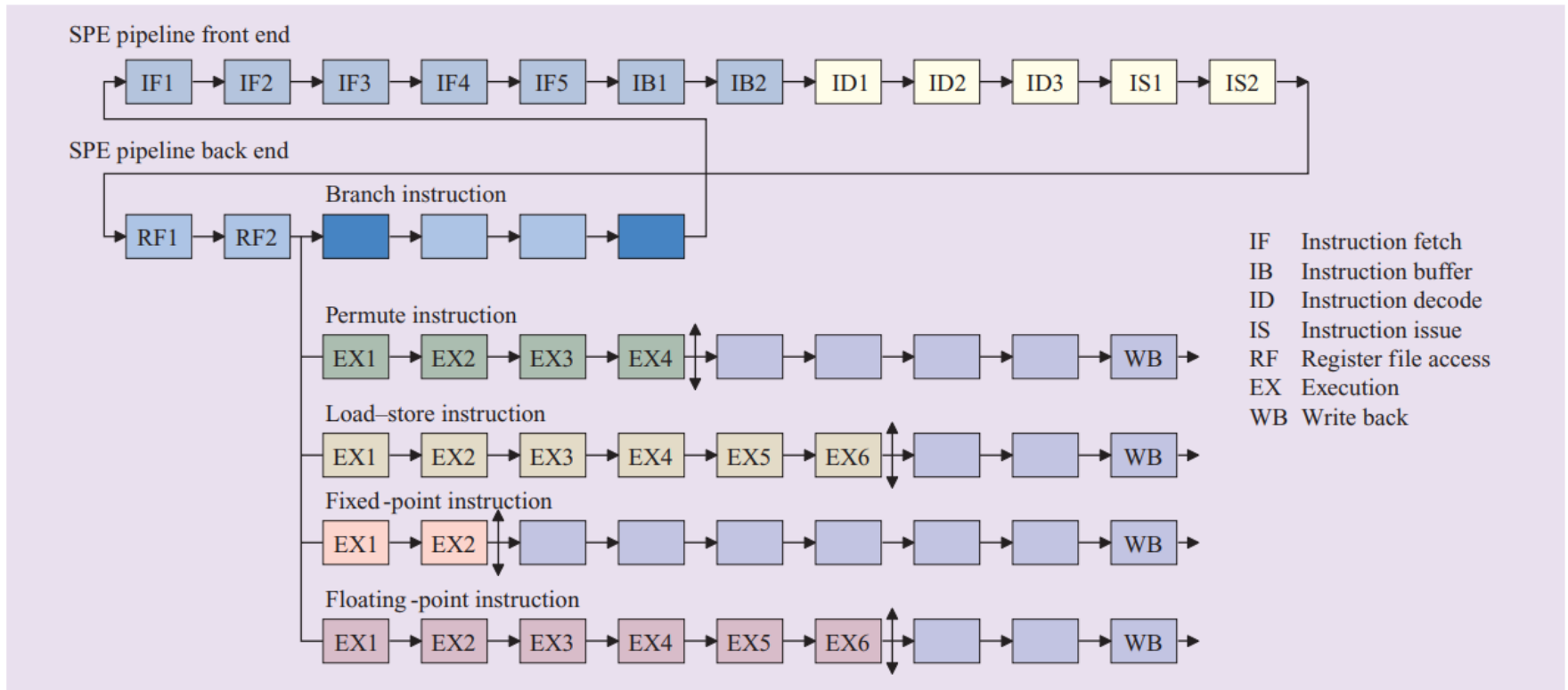
- バックエッジがないパイプライン
 - （回路的な限界にあたるまでは
 - パイプライン段数を増やせば増やすほど性能（周波数）が上がる
- バックエッジがあるパイプライン
 - パイプライン段数を増やすと,
 - 周波数そのものは上がる・・・が,
 - 場合によって, 命令を処理できる実効的な速度が落ちる
 - 特に分岐予測ミス・ペナルティによる性能低下が大きい

余談：実際の CPU のパイプライン段数

- 現在は大体 15 ～ 20 段
- Intel Pentium4 (Prescott) 31 段
 - 2004年発売で 3.8 GHz
 - おそらく、歴史上最大の段数
 - 熱くなりすぎ & 性能が出ずで、この後ステージ数は減少
- AMD Zen : 19 段
 - 2017年発売で 4.2GHz

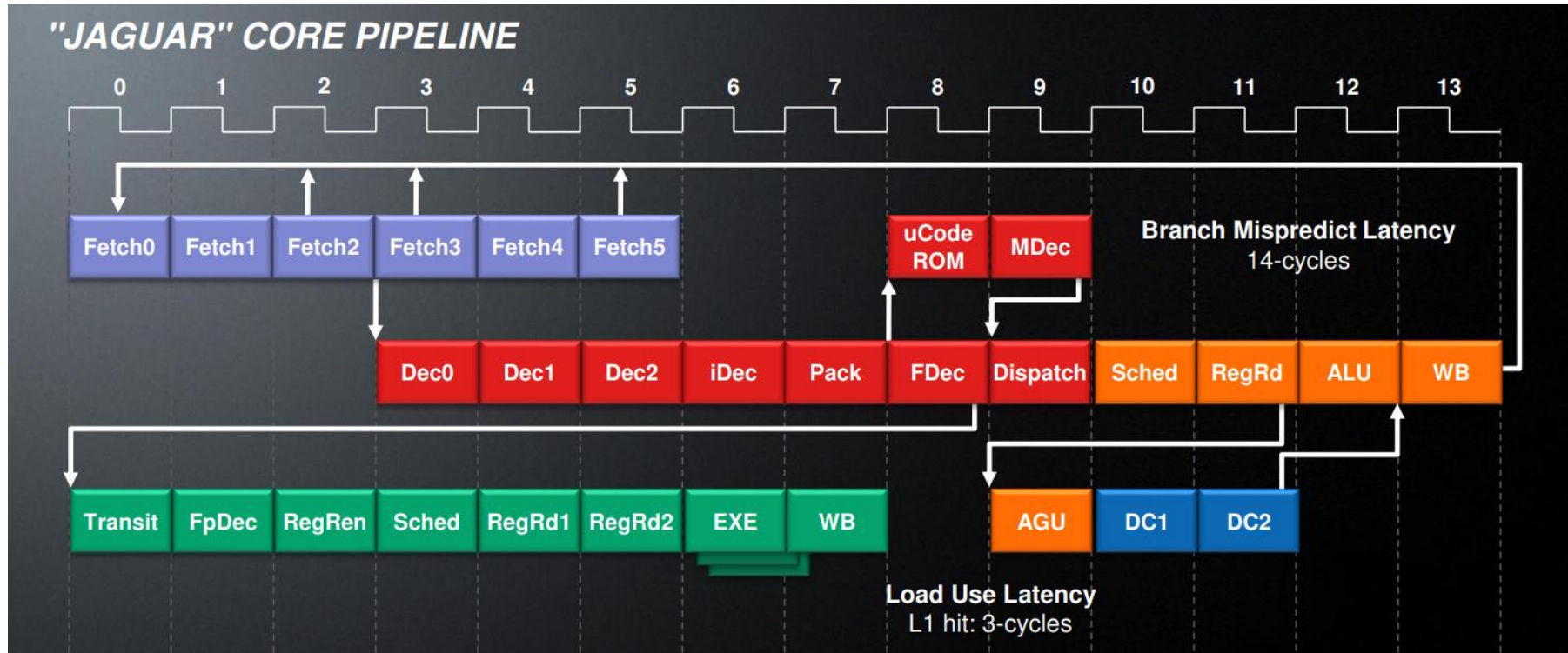
Sony/IBM/東芝 Cell (SPE)

Cell Broadband Engine Architecture and its first implementation—A performance view より



AMD JAGUAR

"JAGUAR" AMD's Next Generation Low Power x86 Core より



ARM Cortex-A15

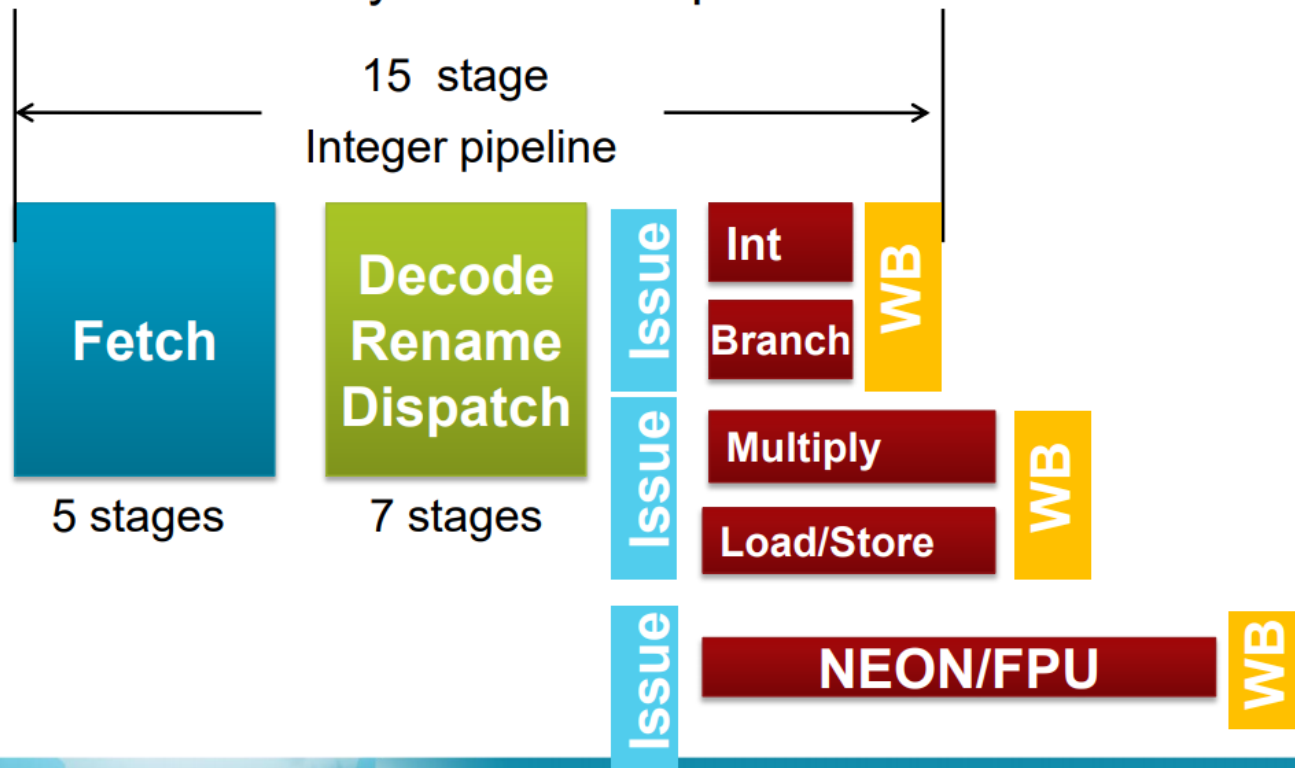
Exploring the Design of the Cortex-A15 Processor

ARM's next generation mobile applications processor より

Cortex-A15 Pipeline Overview

15-Stage Integer Pipeline

- 4 extra cycles for multiply, load/store
- 2-10 extra cycles for complex media instructions



まとめ

1. 構造ハザード：ハード資源の不足に起因
 1. 構造ハザードとはなにか？
 2. その解決方法
2. 非構造ハザード：バックエッジに由来
 - a. データ・ハザード
 - b. 制御ハザード

課題 6

- RISC-V の「add x1←x2,x3」命令を 2 進数で表記すると以下の通りとなる

0000000 00011 00010 000 00001 0110011

(rd←rs1,rs2 として考える)

- (1) 上記を sub x1←x2,x3 に書き換え, 2 進数と 16 進数の双方で表記せよ
- (2) 上記を add x2←x3,x4 に書き換え, 2 進数と 16 進数の双方で表記せよ
- (3) 上記を addi x1←x2,16 に書き換え, 2 進数と 16 進数の双方で表記せよ

- 第 3 回目の講義および次のページの仕様を参考にとすると良い

RISC-V の 基本整数命令

■ 概要

- 加減算, 論理演算, ロード・ストア, 即値, 分岐とジャンプなど
- 各命令は 32bit 幅

RV32I Base Instruction Set

imm[31:12]					rd	0110111	LUI
imm[31:12]					rd	0010111	AUIPC
imm[20:10:11:19:12]					rd	1101111	JAL
imm[11:0]			rs1	000	rd	1100111	JALR
imm[12:10:5]		rs2	rs1	000	imm[4:1:11]	1100011	BEQ
imm[12:10:5]		rs2	rs1	001	imm[4:1:11]	1100011	BNE
imm[12:10:5]		rs2	rs1	100	imm[4:1:11]	1100011	BLT
imm[12:10:5]		rs2	rs1	101	imm[4:1:11]	1100011	BGE
imm[12:10:5]		rs2	rs1	110	imm[4:1:11]	1100011	BLTU
imm[12:10:5]		rs2	rs1	111	imm[4:1:11]	1100011	BGEU
imm[11:0]			rs1	000	rd	0000011	LB
imm[11:0]			rs1	001	rd	0000011	LH
imm[11:0]			rs1	010	rd	0000011	LW
imm[11:0]			rs1	100	rd	0000011	LBU
imm[11:0]			rs1	101	rd	0000011	LHU
imm[11:5]		rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]		rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]		rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]			rs1	000	rd	0010011	ADDI
imm[11:0]			rs1	010	rd	0010011	SLTI
imm[11:0]			rs1	011	rd	0010011	SLTIU
imm[11:0]			rs1	100	rd	0010011	XORI
imm[11:0]			rs1	110	rd	0010011	ORI
imm[11:0]			rs1	111	rd	0010011	ANDI
0000000		shamt	rs1	001	rd	0010011	SLLI
0000000		shamt	rs1	101	rd	0010011	SRLI
0100000		shamt	rs1	101	rd	0010011	SRAI
0000000		rs2	rs1	000	rd	0110011	ADD
0100000		rs2	rs1	000	rd	0110011	SUB
0000000		rs2	rs1	001	rd	0110011	SLL
0000000		rs2	rs1	010	rd	0110011	SLT
0000000		rs2	rs1	011	rd	0110011	SLTU
0000000		rs2	rs1	100	rd	0110011	XOR
0000000		rs2	rs1	101	rd	0110011	SRL
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
0000	pred	succ	00000	000	00000	0001111	FENCE
0000	0000	0000	00000	001	00000	0001111	FENCE.I
0000000000000			00000	000	00000	1110011	ECALL
0000000000001			00000	000	00000	1110011	EBREAK
csr			rs1	001	rd	1110011	CSRRW
csr			rs1	010	rd	1110011	CSRRS
csr			rs1	011	rd	1110011	CSRRC
csr			zimm	101	rd	1110011	CSRRWI
csr			zimm	110	rd	1110011	CSRRSI
csr			zimm	111	rd	1110011	CSRRCI

画像は下記より

The RISC-V Instruction Set Manual Volume I: User-Level ISA Document Version 2.2

提出方法

■ 以下を提出：

1. 課題 6：

- 提出は Moodle の「課題 6」のところからお願いします

2. 感想や質問：

- 「感想や質問」のところに投稿してください
- わからない場所がある場合，具体的に書いてもらえると良いです

■ 提出締め切り

- 次回講義開始まで

■ 注意：

- 課題の出来は，ある程度努力したあとがあれば良しです
 - 必ずしも正解していなくても良いです
- 課題は成績の判定にかなり使われます
 - 仮に課題を一度も出さなかった場合，
期末試験だけ受けてもまず通らないと思います

質問や感想など

質問や感想など

- また、コンピュータの内部は回路であるので、講義で何か電子回路を自分で組んで何かやる実験をしたいなとふと思いました。（昔、電気通信大学の高校生向けプログラムでやって、面白かったことを思い出した。）
- うちの研究室内の CPU を実際につくる演習です：
 - <https://github.com/shioya-lab/cpu-exercise>
 - もともとは名古屋大学の3年生むけ学生実験でした
 - 大半はシミュレーションで実施するので、手元の PC で出来るはず
 - Mac の場合はちょっとめんどくさいかも
 - ◇ 本当にやりたい人がいたら相談してください

質問や感想など

- 今回の講義資料がダウンロードできずとても閲覧しにくかったので Moodleでの講義資料の掲載を検討していただけると幸いです。
- ちょっとわかりづらいかも知ですが、ダウンロードできるはずです



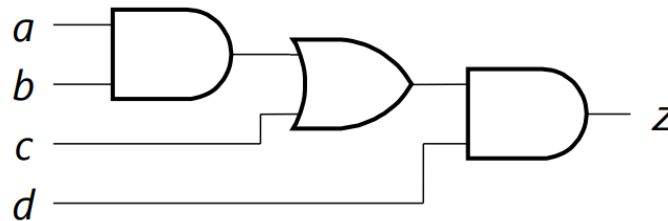
- 今回の課題もより一層テストが不安になりました。

- パイプライン段数を増やしていても無限に早くなるわけではない
というのを聞いて、どこかで聞いた「プログラマーの数を増やせば
反比例でプログラム作成にかかる日数が短くなるわけではない」と
いう話を思い出した。

- パイプライン方式では、仮想的にラインを複数同時に扱うために、コストやスペースが多く必要になったりしないのでしょうか。

質問や感想など

- 講義資料39ページのような図は飽きるほど書いたことがあるのですが、課題であるようなP型、N型で作られるNANDゲートは見覚えがないので課題があっているか不安です。



- コンピュータに使われているようなCPUは3,4個並行して処理しているとのことですが、それは高速に3,4個の処理を順番に行なっているのですか？

- コンデンサーを高校物理で習っていたが、ここで使われるとは思わなかったです。習ってる時は何を習っているのか、これがなんの役に立つのかもわからなかったのですが、確かに初見でこれをやるよりは前提知識があるので高校で物理を選択していて良かったです。

質問や感想など

- 大学に入学したばかりの頃、パソコンを購入する際に「クロック周波数」という項目があるのを見かけましたが、そのときは意味が分からず、特に気にせず選んでいました。
- 今回の授業でパイプライン化やクロック周期、クロック周波数について学ぶ中で、あのときのクロック周波数はこのことだったのか、と気づき、とても興味深く感じました。

質問や感想など

- 今回の内容はかなりすんなりと理解できました。ようやく去年勉強したこととこの授業で学んだことが身についてきた感じがします
(涙)

- 最後に紹介されていたゲームをやりたいと思ったのですが日本語版がなくて怖気付きました。
 - たぶん、英語がそんなにわからなくても大丈夫だと思う

質問や感想など

- 課題に取り組む中で思ったのですが、P型とN型がどちらも閉じていて直列の時の値はどうなりますか？また、有り得ない組み合わせかたはありますか？
 - どちらも閉じる状態が生じないように回路を組みます
 - =どちらも同時に閉じる状態があるなら、何か間違ってる

- パイプライン化では1つのステージで問題が起きて処理が完了されなくなった時、すでにそのステージを超えている命令の処理もラッチによって止まってしまうのは問題にならないのかなと思いました。(例えば命令I1のIFでの処理が止まった時すでにIDのステージにある命令I0の処理も止まってしまうみたいな)

- パイプラインの段数を増やすことと速度の関係が、アキレスと亀の問題みたいだと思った。

- パイプライン化の話が難しく、雰囲気を読む程度しかできませんでした。今後の課題や授業を通して、少しずつ理解していけるといいなと思います。

質問や感想など

- P型/N型リレーについて、前回理解した気になっていたのですが、実際に今回の課題を解いたことで、自分が理解していない部分が見えてきたので良かったです。課題をやることでやっと理解するという事が多く、演習問題のようなものがあれば解きたいなと思うので、この授業の内容と合った問題が載っている本やサイトがあったら教えていただけると幸いです。
- とりあえず講義のページにある演習問題をやってみてください

- 今回の課題のようなP型/N型リレーを使って書く問題の場合、スイッチのオンオフは、好きな入力例(例えば入力aが1で入力bが0)を選んで書いていいのでしょうか？

質問や感想など

- レイテンシとスループットについての話を聞いた時は、他の授業でパイプライン化について学んだ際にパイプライン化はただ処理を速くするものという認識をしていたため、レイテンシは短くならないどころか逆にやや延びることもあったと知って驚いた。
- また、パイプライン化によって、一続きの処理における処理時間は長くなることもあるが命令数に着目すると単位時間当たりの処理命令数が増えるということは、処理する命令が多くなるにつれてパイプライン化の効果は現れやすくなり、より効果的な手段であると言えるようになるということなのかと疑問に思った。

- ただ、私は知識が浅く、シングル・サイクル・プロセッサの動作の説明が漠然としか理解できなかったもので、もう一度調べながら復習しようと思います。

- 課題5のような問題について、スイッチの開閉状態は真理値表のうちどの行を表す状態で書いても良いのでしょうか。普通この行で書くというような基本はありますか？
 - どれかの行に対応していれば、それで良いです

質問や感想など

- 数字をレジスタに入れるか数字のまま書くかによって何か違いは出るのでしょうか。どちらに統一した方がよいなどのルールはあればご教示いただきたいです。
 - 数字のまま書く（即値）と命令数が減るので楽です
 - プログラム中で計算される値は即値に出来ないので注意

- 第4回課題について、 B に0を代入する必要があるのはなぜですか。最後の行で、 $A > 0$ としてはいけないのですか。

- 命令パイプラインを見ていて、カエルの合唱の輪唱をしているようだと感じた。ラッチによってそれぞれの仕事が溜まったり、滞ったりしないようにするという仕組みは、興味深かった。人間の工場ラインと似ているような部分も多く、理解しやすかった。

質問や感想など

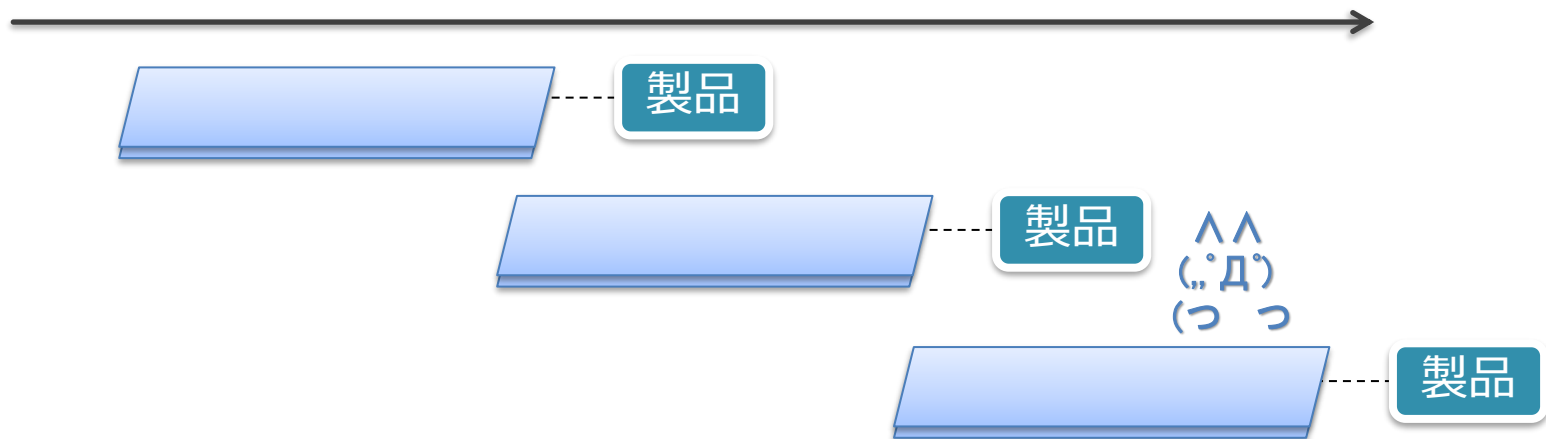
- パイプライン化において、作業する人の分だけ処理が高速になるという話がありましたが、高速化の程度は何を基準にしてはかられているのか疑問に感じました。

というのは、第5回の資料の15ページの図を見ると、パイプライン化しなかった場合は1人で製品を全て作っているためトータルでステージ12個分の時間がかかり、パイプライン化した場合は製品を作るプロセスを4つのステージに分けて4人で作業を並行して行なっていてトータルではステージ6個分の時間がかかっています。結果、作業する人数は4倍になりましたが、同じ量の製品を作り終えるまでにかかった時間は2倍になっています。このとき、パイプライン化によって性能が2倍に向上したという認識でいいのでしょうか。それとも、時間以外の何かしらの基準に基づいて処理の速度が何倍になったかを決めているのでしょうか。

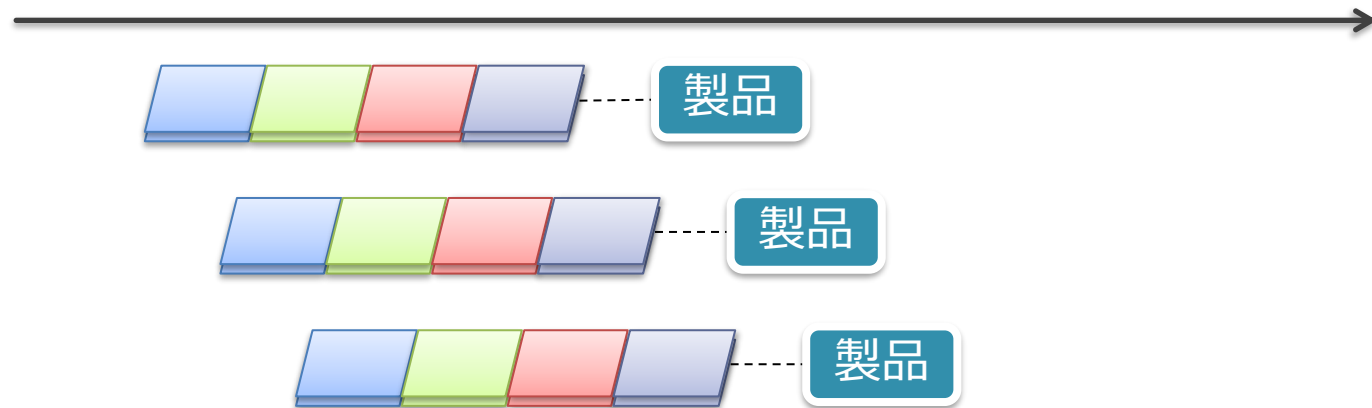
- 製品の個数が少ないときは「レイテンシ」が見えてしまう
- 無限個の製品を作ったときを考えると「スループット」が何倍になったのかがわかる

パイプライン化による性能向上

パイプライン化しない場合



パイプライン化した場合



質問や感想など

- 期末試験について、どのような形式で問題が出題されるのかなどを予め把握できた方が勉強しやすいように感じるので、過去問や出題例などを提示していただけるとありがたいです。
- 基本的に課題と演習問題が解けるようになっていれば大丈夫です

質問や感想など

- 期末テスト対策としてどんな練習をすれば今回の課題のような論理ゲートをスラスラかけるようになるでしょうか？やはり色々な論理ゲートを調べて覚えることでしょうか？

質問や感想など

- 先生が授業で紹介されていた「実際のCPUを実行した場合のパイプライン」について、今まで簡略化されたものしか見たことがなかったので興味深かったです。なお、色付けされているのは、見やすいように後から先生が編集したものなのか気になりました。
- <https://github.com/shioyadan/Konata>
- これは私が作ったツールで、こういう色を出すようにしています
 - 結構いろいろな会社で開発に使われています

Instruction	Dc	Sch	Xl	Wku	X
1	1	2	3	4	5
2	1	2	3	4	5
3	1	2	3	4	5
4	1	2	3	4	5
5	1	2	3	4	5
6	1	2	3	4	5
7	1	2	3	4	5
8	1	2	3	4	5
9	1	2	3	4	5
10	1	2	3	4	5
11	1	2	3	4	5
12	1	2	3	4	5
13	1	2	3	4	5
14	1	2	3	4	5
15	1	2	3	4	5
16	1	2	3	4	5
17	1	2	3	4	5

Instruction	Dc	Sch	Xl	Wku	X
1	1	2	3	4	5
2	1	2	3	4	5
3	1	2	3	4	5
4	1	2	3	4	5
5	1	2	3	4	5
6	1	2	3	4	5
7	1	2	3	4	5
8	1	2	3	4	5
9	1	2	3	4	5
10	1	2	3	4	5
11	1	2	3	4	5
12	1	2	3	4	5
13	1	2	3	4	5
14	1	2	3	4	5
15	1	2	3	4	5
16	1	2	3	4	5
17	1	2	3	4	5

質問や感想など

- 今回の授業のスライドはアニメーションでとてもわかりやすかったです。そこはかたなく時代を感じるアスキーアートも可愛かったです。
- 余談ですが、スライドの絵文字芸に癒されています。
- 資料の猫が可愛くて分かりやすかったです

^^
(.Д.)
(つ つ

^_^
('▽')
(つ つ

^^.
(*ー)
(つ つ

^_^
('ω')
(つ つ